Data Driven Mobile Fitness App

Table of Contents

Analysis

Describing the Problem

The problem is that there is a lack of fitness apps that give users clear, data driven feedback about their progress and gives adaptive goals for them to complete.

The app will be at first developed for weightlifters with android phones but will hopefully be expanded to include IOS users.

The app will allow users to store information locally about workouts that they complete which includes weight lifted for how many repetitions (reps) on a specific movement. Then the app will produce a line graph for each exercise that the user has completed which shows their progress in that exercise. At the beginning of each month, the app will generate an adaptive goal for each exercise to reach by the end of the month based on the trend shown by the graph it has stored for that exercise. The app will be developed so that user-defined routines can be quickly recorded and will also track additional data such as bodyweight and consistency in the gym (the latter of which can be compared to the user's goal for that week). Reviewing exercise data will also be made easier by allowing the user to pin exercises to the homepage.

All information will be stored locally so each installation of the app will only cater for one user.

Research

In my research section I aim to gather information from subject experts, current solutions and anything else that I will need to know before I start programming. At the end of the section I will describe how my research will affect my objectives.

Expert Interview

Mr Fisher is a trained Strength and Conditioning Coach that works at the school and is the main manager and frequent user of the school gym. I thought it would be appropriate to interview him as an expert in the area, considering he develops programs for many of the young athletes at the school.

This was the brief I gave to Mr. Fisher before the interview:

> "I'm making a fitness app primarily for recording weightlifting workouts but if I have time at the end will include cardio workouts. Then the app will draw some graphs to show how you've improved and then will create some goals for you to hit for each exercise."

I have highlighted Mr. Fisher's key points.

Dialogue

So, have you ever used a fitness app before, for recording workouts?

> "Yeah Several. Although, I don't think I've used like the best ones in terms of like load management and predicted goals like you're talking about. The ones I've used haven't been able to record weights.
>
> That's never really been my goal, but [in the app ATG Online Coaching] you can send in a video of yourself and then a coach gives you feedback on the shape, you're making and stuff right. What else did it do? You could also do Q & A with coaches about any questions you had about the [workouts that they suggested], which was quite helpful. Yeah, it had like some technical experts on the other side, which was which was, which is quite nice. But I know that

there are some pretty useful apps. And I think there's TeamBuildr, and there's Output Capture that both record data and use it in quite helpful ways."

Were there any kind of issues with those apps that you found?

"I think the user experience, for most of the apps that that I've seen used, it's been pretty good. I think TeamBuildr is super easy to use and it was easy for coaches to use and it's easy for athletes to sort of reorganise and understand what they're doing.

The only problem I found, I mean, one of the reasons I didn't use TeamBuildr, is because the scheduling can be not very flexible so it's kind of limited an athlete's choice on what they were doing on a certain date which is potentially a good thing, and a bad thing.

So, there wasn't that much flexibility in people schedules which wasn't great. One of the biggest downsides, just from a coach's point of view, [is that] learning to navigate the system is a big cost in time. Yeah, so just making that that front-end experience to set things up for your athletes, for the people you're coaching making that really seamless is actually quite tricky and potentially daunting for people, right?"

Do you think that the computer-generated goals would be helpful?

"It depends, I think. I mean, if that's the only variable people want, then there are lots of ways to go about that. I think rep max formulae [more information here https://en.wikipedia.org/wiki/One-repetition_maximum] have some serious limitations.

I think the physiological variables in the body can affect performance in so many ways. Especially in a school setting where so many variables are kind of wavering that … might not be the optimal way of way of working.

It could be useful if your goals are simply weight driven, but even that might be tricky because progress is never linear and it's different depending on your training age and your levels of relative strength. So, it can be useful, but not always. It depends on [your motivation a lot] as some people are intrinsically motivated by moving the weight."

Key Takeaways from the Interview

- UI (User Interface) is important
- Must be easy to learn how to use
- Flexibility
- There are limitations for estimated one rep max (e1RMax)
- Research Teambuildr and Output Capture

One Rep Max Formulae

p. 69 in Program Listing

In order to get a graphical representation of progress at the gym over time, I need some kind of metric to measure how 'good' a certain set (an exercise completed for a certain number of

repetitions at a certain weight) is. In our interview, Mr Fisher mentioned the One Rep Max Formulae which are ways of estimating somebody's one rep maximum (1RM) or how much weight someone could lift for one rep with good form based off a single set.

I have since researched them and used a graphing calculator to look at the differences between them (below). He did mention that the formulae have limitations so I will try to put an emphasis on looking at long term trends, as to minimise the environmental effects he talked about.

Since it is used extensively in the write up and code listing, **estimated One Rep Maximum** will often be shortened to **e1RMax** or **e1RM**.

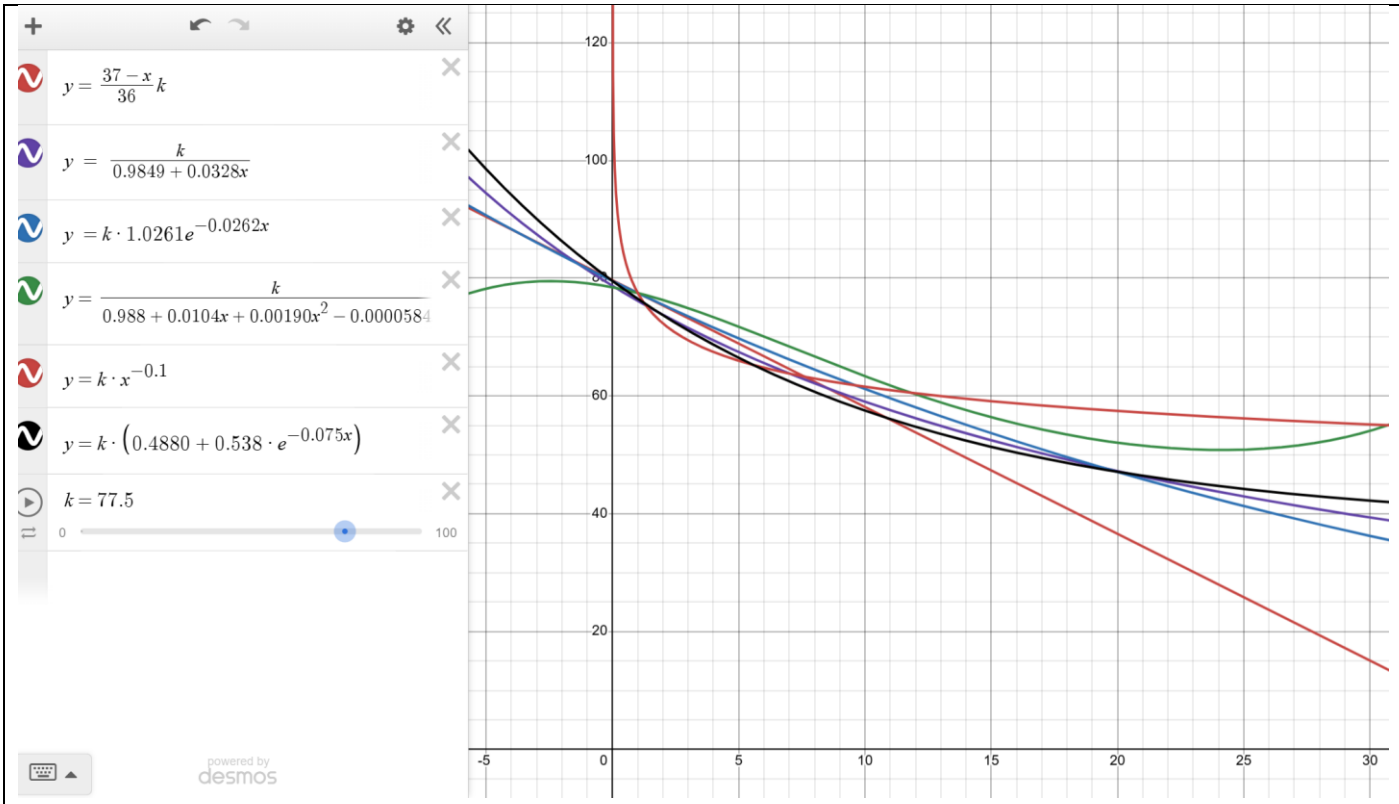


Figure 1: A Variety of One Rep Max Formulae

Here are several different models (Wikipedia, 2024) for the relationship between number of repetitions (x axis) and the predicted weight lifted (y axis) for an exercise with a given 1 rep max (the constant k).

Then I did some research using data from Mr JD Thomas and myself to determine that the Brzyki formula (w = 1RM * (37 - r)/36) is more accurate for 1 <= r < 8 and the Lombardi formula (w = 1RMax * r^-0.1) is more accurate for r < 1 and r >= 8.

Part of the selection process also included eliminated the ones that don't pass through (1, k) so that if somebody records an actual one rep max, it is properly recorded.

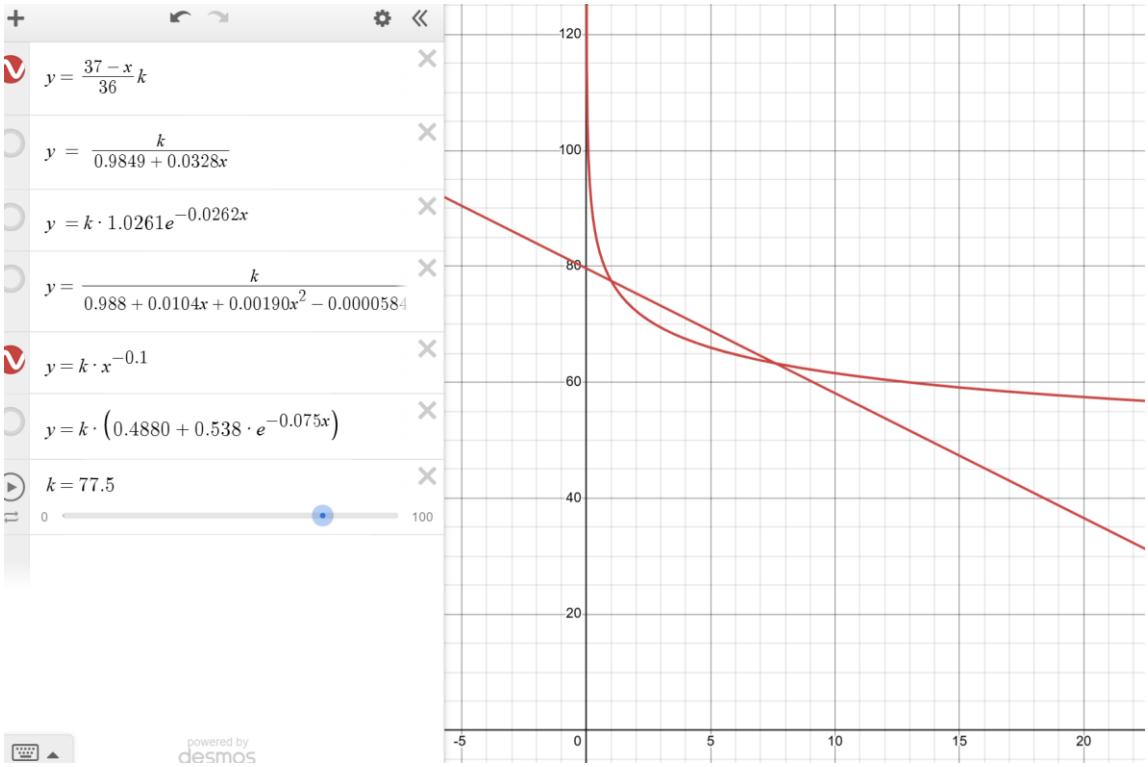
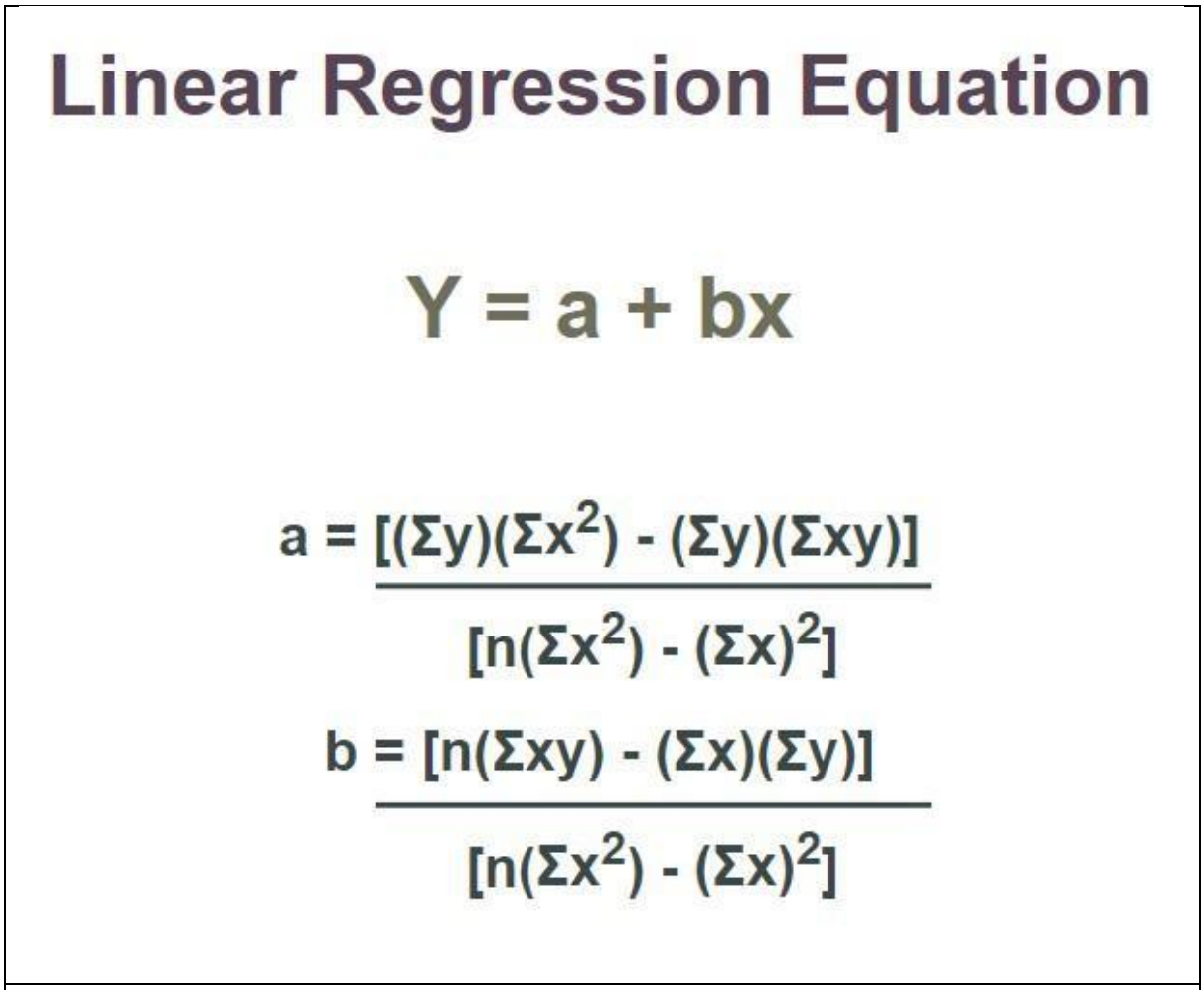


Figure 2: The Selected Formulas

The decision for using different formulae for different rep ranges, came not only from my research data but also from looking at the interceptions between the two graphs (which have the same x values irrespective of k). It made sense to transition between the different formulae where the difference between them was at its lowest. It also made sense to use the linear model in the middle section as logically speaking, the best model for the extremes shouldn't cross either axis.

Regression Formula



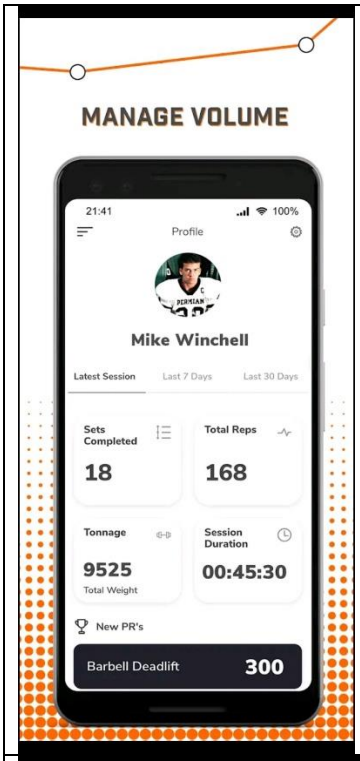
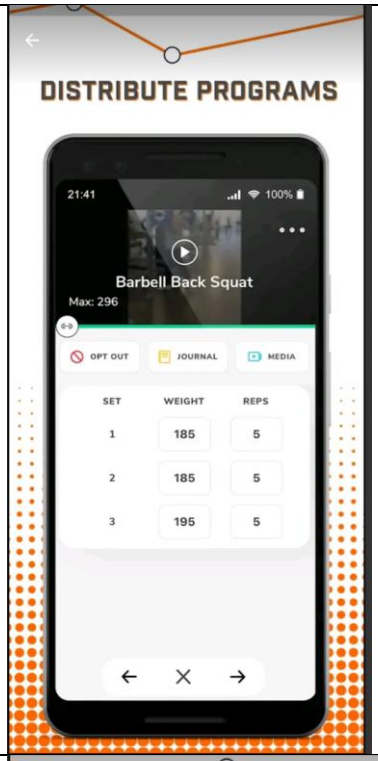
Source: Geeks for Geeks (2024)

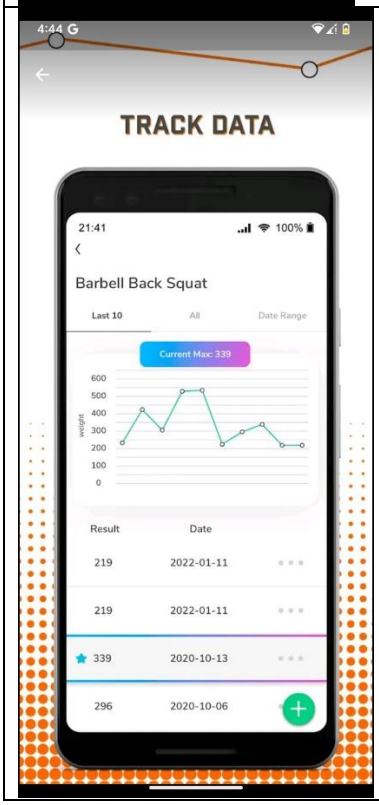
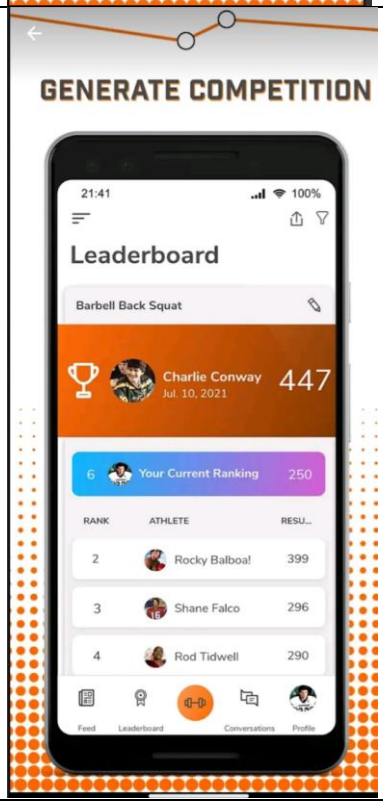
pp. 55-56 in Program Listing

To generate a goal for a certain exercise, I will need to apply this formula to the data with y representing estimated One Rep Max and x representing time. This will get me the line equation for the regression line, which can be used to find a prediction for the user's future estimated one rep max by subbing in a value for x.

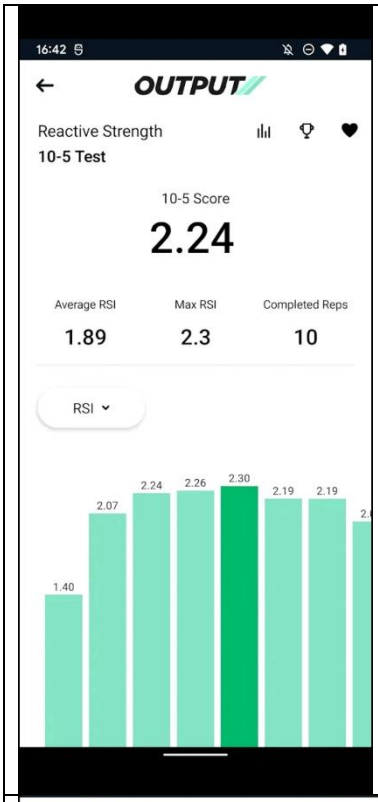
Existing Tools

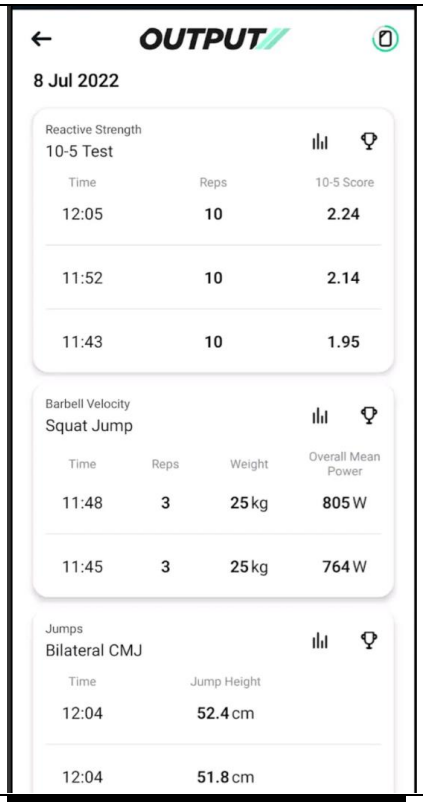
TeamBuildr

<table>
<tr>
<td> 
</td>
<td> 
</td>
<td rowspan="2">Pros
• Good UI
• Graph is helpful (last 10 sets time control is good)
• Current Max is helpful.

Cons
• Tonnage & total reps aren’t that relevant
• The app has a slightly different purpose and is more for managing teams of athletes as the name suggests.</td>
</tr>
<tr>
<td> 
</td>
<td> 
</td>
</tr>
</table>

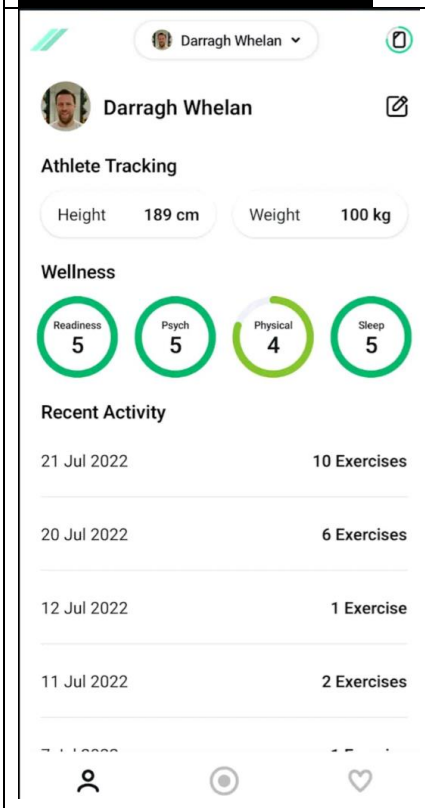
Output Capture

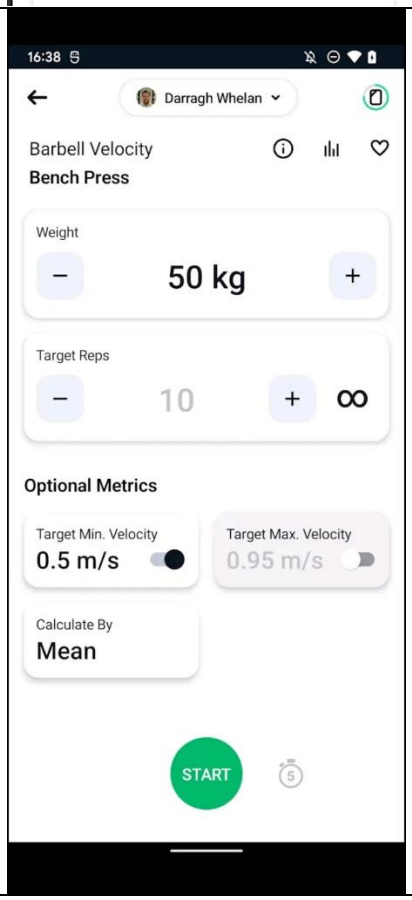




Pros

- The bar chart is helpful.
- Recent activity is helpful.
- Wellness ratings are interesting.
- Calculates target reps for a given weight.

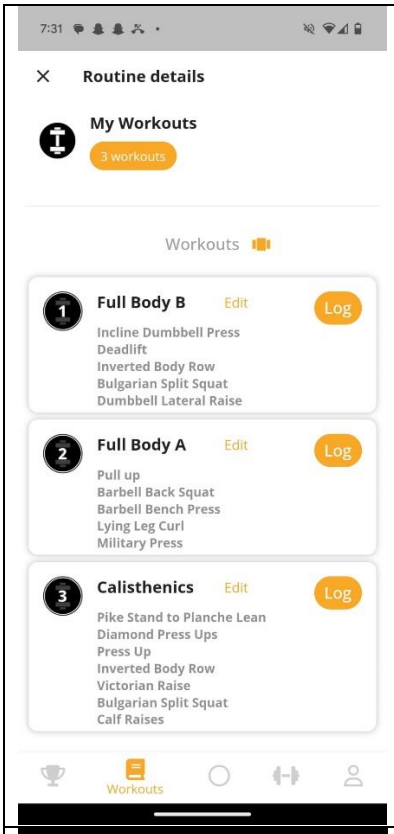


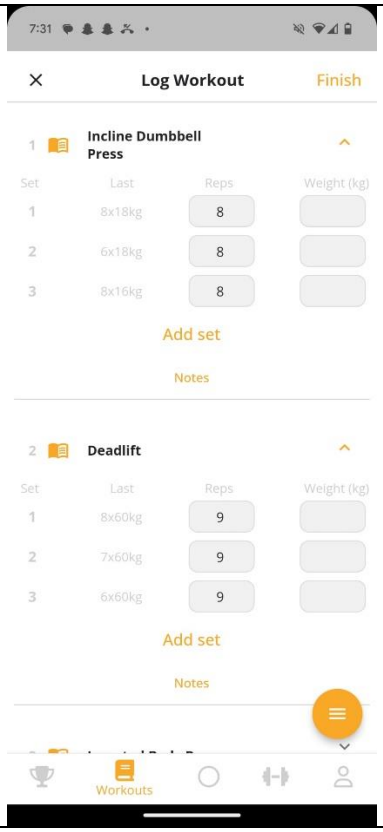


Cons

- A lot of the information on the app is not very intuitive, to me at least.

Stronger





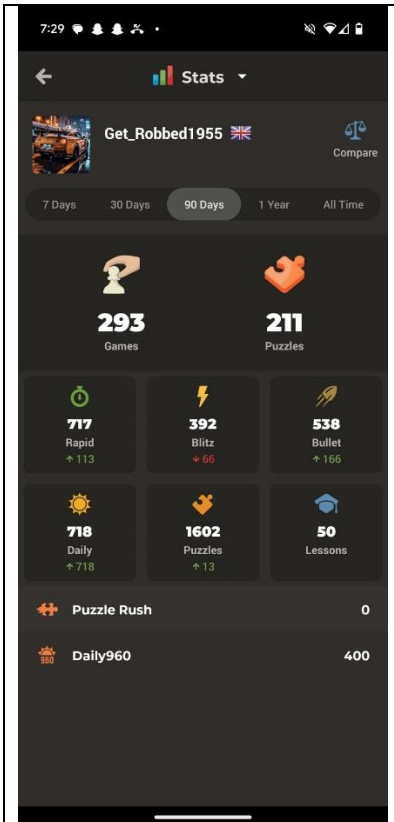
Pros:

- Weight milestones are an interesting touch.
- It's useful to define own exercises if your exercises are obscure.
- Records personal bests (PB)
- "Snapped" feature is useful, tells you how consistent you have been with your workouts and how many PBs you have achieved since starting on the app.
- Weekly challenges are nice e.g. how many gorillas, rhinos etc. can you lift in a week.

Cons:

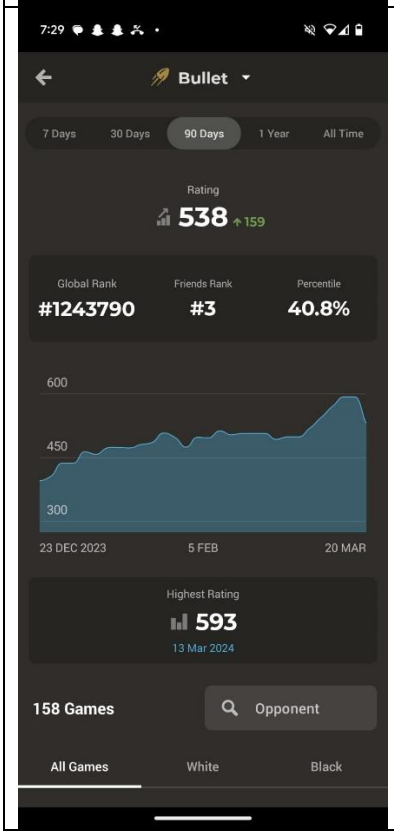
- Lots of clutter
- No numerical analysis or use of graphs
- Limited access to history
- At high levels the milestones become arbitrarily large numbers, which is a problem as it takes up quite a lot of real estate on the app.
- Social aspect of the app is very poor.
    - Posted workouts receive little interaction most likely because theirs no way of indicating an author's credibility.

Chess.com



Pros:

- I really like the UI surrounding how the data is laid out.
- The graph is useful & transferrable to my project.
- The highest rating is useful & transferrable.



Cons:

- The absolute change in rating between now and X days ago is not as useful and potentially quite demotivating if it is negative.
- Not exercise based, will have to make some modifications to features to make them meaningful.

Implementation Research

Firstly, I needed to choose a cross-platform UI framework if I wanted to develop for both IOS and Android. I ended up choosing Xamarin Forms for its popularity, ease of use and my preference for writing in C# as opposed to Python.

I also needed a way to implement a line graph to display data as there is not a way to do this in base Xamarin. I ended up acquiring a free developer's license for Syncfusion, an extension to Xamarin that allows me to use a chart tag in the markup language for Xamarin, XAML. I discovered Syncfusion through a Microsoft dev blog (Montemagno 2015), more details can be found in my acknowledgments section.

Choosing an Emulator

In order to test my application, I would need to use an emulator. I decided on Microsoft's android emulator since I primarily wanted to develop for android and also because it can be integrated into visual studio.

Impacts of my Research on my Objectives

Through my interview and examination of existing fitness apps, I have determined that making sure the app is simple and intuitive to use is a priority. If there are too many functions that don't suit the core needs of the user, it clutters the app and makes the UI overly confusing. Therefore, I will try to write my objectives such that I focus on creating an app that provides clear, data driven feedback on the user's gym progress. I will carefully consider every potential function of the app so that I can focus on the quality and complexity of them.

Additionally, when I write the objectives, I will check that Xamarin Forms and Syncfusion can implement everything I want included in the App.

Furthermore, I have decided that the main metric for the user's progress will be estimate one rep max, if it be based on a single set or a month of sets. I have been told by Mr Fisher that it has limitations, so I will put a focus on long term trends e.g. using goals that are predictions for a long time in the future (3 months).

System Objectives

High Level Objectives

- The user will be able to store records of their workouts in a database.
- The app will then display this data in a way that means the user can see how they have been performing in the gym recently and where they might be in the future.
- This will be implemented by showing graphs of estimated 1 rep max for each set against time for a given exercise, personal best set for a given exercise, a goal for a given exercise and a consistency rating measured in average sets per week.
- The database will be maintained by taking medians within certain time periods, in order to exclude outliers from the data.
- The database will also store the User's bodyweight (in kilograms) and consistency at the gym (in number of sets per week), both of which can be displayed on graphs.

Full Objectives List

Here are the complete objectives of the system, ordered roughly according to the critical path of the project and divided into the 3 main stages of development.

Stage I

1. App works for android phones.
2. User can navigate intuitively between different pages of the app.
3. Tables for Pending Sets, Monthly Medians and Daily Medians implemented using an SQL Database.
4. User can store a set by inputting the weightlifting exercise, weight used, and repetitions (reps).
5. Based off the user input for a set, the app stores locally the type of exercise, date and a prediction for how much weight that person could lift for 1 repetition (rep), as a marker for how powerful that set was (referred to as estimated 1 rep max or e1RM).
6. User can view sets from the last month in a table.
7. Database stores, in a separate table, medians of e1RM, to smooth out the graphs and make trends more obvious.
    a) For each exercise, the app stores the median e1RM for sets on the same day, on days this month.
    b) For each exercise, the app stores the median e1RM for sets in the same month, once that month has ended.
8. User can view graphs of median e1RM (both daily and monthly) against Time for a given exercise.

Stage II

9. Implement tables for User Data, Exercises, Routines and Routine Components in the previously mentioned SQL database.
10. User can record their bodyweight in the app.
11. App stores weight of user over time.
12. User can define custom exercises, inputting their name and whether they are bodyweight exercises or not.
13. App accommodates the recording of bodyweight exercises by automatically adding the stored weight of user to the weight inputted.
14. User can define their own workout routines, by inputting a list of exercises and the number of sets for each of those exercises.
15. User can log user-defined workout routines quickly, by loading up a workout then inputting, for each set, the number of reps and the weight.
16. When logging a predefined workout, the User can deviate from their defined workout by adding or removing exercises and/or sets on the log workouts page, before they log the workout.

Stage III

17. App can generate a goal for each exercise for the next 3 months, based off their performance in that exercise over the last 3 months.
18. App stores a personal best (PB) e1RM for each exercise.
19. User can pin an exercise's PB and Goal to the welcome page.
20. User can view information about each exercise in a table with the columns: Exercise, PB, Goal and Is Pinned.
21. User can undo/redo the adding of sets recorded in the current session

22. User can define a goal for weekly consistency, measured in total sets per week (across all exercises)
23. User can compare last week's consistency to the goal set for that week.
24. User can view graphs of their bodyweight and weekly consistency over time.
25. App works for IOS.

Reasoning Behind Certain Objectives

5. Based off the user input for a set, the app stores locally the type of exercise, date and a prediction for how much weight that person could lift for 1 repetition (rep), as a marker for how powerful that set was (referred to as estimated 1 rep max or e1RM).

The reason I am storing estimated One Rep Max instead of storing the reps and weight separately and then calculating whenever the data is retrieved is twofold:

- There are several occasions where I might need to order my select statements by e1RM and since it is not possible to do the calculation within the SQL statement, I would have to sort the results with a separate algorithm. This would impair efficiency as well as take time away from other parts of the program.
- The other reason is that I have no reason to recall the number of reps or weight of a particular set. Storing reps and weight would be making the code unnecessarily convoluted

17. App can generate a goal for each exercise for the next 3 months, based off their performance in that exercise over the last 3 months.

I chose to generate goals for 3 months in the future because it is a long enough time to minimise the short-term variations that Mr Fisher talked about in my interview, but short enough so that it is in sight for the user as something to aim for. I chose to back date the data that the prediction is based off by 3 months because any more than that, I would expect that the natural curvature of progress in the gym, over long periods, would make assuming a linear progression inaccurate.

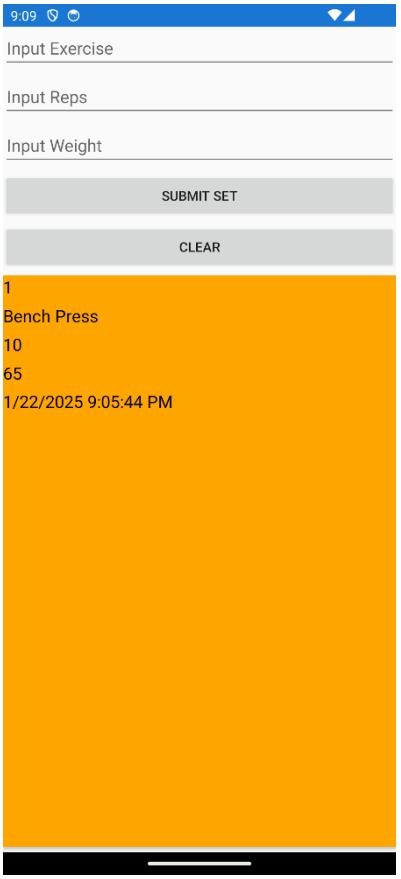
18. App stores a personal best (PB) e1RM for each exercise.

I was initially unsure whether personal best should represent the highest weight ever actually inputted, or the highest estimated 1 rep max calculated for that exercise. Eventually, I chose the latter because I wouldn't want to exclude people who only practiced high repetition training, of which I know many, from this feature.

Initial Modelling

Prototype

As you can see, I prototyped my app as a single page application with a simple database, input form and table display. I experimented with trying to program my own navigation system, but I found that it would be too complicated to code myself which would detract from the rest of the project. Therefore, I concluded that I needed to use one of the navigation templates offered when creating a new Xamarin Forms project on Visual Studio. I opted for a flyout template because of the modularity that it offered if I wanted to create more pages.



App.xaml.cs | Database.cs | testTable.cs | AddSets.xaml | AddSets.xaml.cs

Fitness App | Fitness_App.App | OnStart()

```
using System;
using System.IO;
using Xamarin.Forms;

namespace Fitness_App
{
    12 references
    public partial class App : Application
    {
        public static Database database;
        6 references
        public static Database Database
        {
            get
            {
                if (database == null)
                {
                    database = new Database(Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.LocalApplicationData), "FitnessApp.db3"));
                }
                return database;
            }
        }
        2 references
        public App()
        {
            InitializeComponent();

            MainPage = new AddSets();
        }
        0 references
        protected override void OnStart()
        {
        }

        0 references
        protected override void OnSleep()
        {
        }

        0 references
        protected override void OnResume()
        {
        }
    }
}
```

App.xaml.cs | Database.cs | testTable.cs | AddSets.xaml | AddSets.xaml.cs

Fitness App | Fitness_App.Database | SaveTestDataAsync(testTable testSaveData

```
using System;
using System.Collections.Generic;
using System.Text;
using System.Threading.Tasks;
using SQLite;

namespace Fitness_App
{
    4 references
    public class Database
    {
        private readonly SQLiteAsyncConnection _dbcon; // create database connection obj
        1 reference
        public Database(string dbPath) // Database class constructor
        {
            _dbcon = new SQLiteAsyncConnection(dbPath);
            _dbcon.CreateTableAsync<testTable>();
        }
        3 references
        public Task<List<testTable>> GetTestDataAsync()
        {
            return _dbcon.Table<testTable>().ToListAsync(); //returns db as list
        }
        1 reference
        public async Task<testTable> GetTestItemAsync(int id)
        {
            return await _dbcon.Table<testTable>().Where(i => i.Id == id).FirstOrDefaultAsync(); // i => i.Id == id is a lambda function with a comparson function in it
        }
        1 reference
        public Task<int> SaveTestDataAsync(testTable testSaveData) //stores testTable object (aka a record from testTable class) in table, returns new PK
        {
            return _dbcon.InsertAsync(testSaveData);
        }
        1 reference
        public async Task<int> DeleteTestDataAsync(testTable deleteItem) {
            return await _dbcon.DeleteAsync(deleteItem);
        }
    }
}
```

App.xaml.cs | Database.cs | testTable.cs | AddSets.xaml | AddSets.xaml.cs

Fitness App | Fitness_App.testTable | XcizeAttribute

```
using SQLite;
using System;

namespace Fitness_App
{
    8 references
    public class testTable
    {
        [PrimaryKey, AutoIncrement]
        1 reference
        public int Id { get; set; }
        1 reference
        public string XcizeAttribute { get; set; }
        1 reference
        public int RepsAttribute { get; set; }
        1 reference
        public float WeightAttribute { get; set; }
        1 reference
        public DateTime DateAttribute { get; set; }
    }
}
```

App.xaml.cs | Database.cs | testTable.cs | AddSets.xaml | AddSets.xaml.cs

ContentPage | ContentPage

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="Fitness_App.AddSets">

    <StackLayout>
        <Grid RowDefinitions="Auto,Auto,Auto,Auto,Auto,*,Auto">
        <Entry x:Name="xcizeInput" Placeholder="Input Exercise" Grid.Row="0"/>
        <Entry x:Name="repsInput" Placeholder="Input Reps" Keyboard="Numeric" Grid.Row="1"/>
        <Entry x:Name="weightInput" Placeholder="Input Weight" Keyboard="Numeric" Grid.Row="2"/>
        <Button x:Name="submitSet" Text="Submit Set" Clicked="submitSet_Clicked" Grid.Row="3"/>
        <Button x:Name="clearInputs" Text="Clear" Clicked="clearInputs_Clicked" Grid.Row="4"/>
            <CollectionView x:Name="testDataView" Grid.Row="5" BackgroundColor="Orange">
                <CollectionView.ItemTemplate>
                    <DataTemplate>
                        <StackLayout>
                            <Label Text="{Binding Id}"
                               TextColor="Black"
                               FontSize="Medium"/>
                            <Label Text="{Binding XcizeAttribute}"
                               TextColor="Black"
                               FontSize="Medium"/>
                            <Label Text="{Binding RepsAttribute}"
                               TextColor="Black"
                               FontSize="Medium"/>
                            <Label Text="{Binding WeightAttribute}"
                               TextColor="Black"
                               FontSize="Medium"/>
                            <Label Text="{Binding DateAttribute}"
                               TextColor="Black"
                               FontSize="Medium"/>

                        </StackLayout>
                    </DataTemplate>
                </CollectionView.ItemTemplate>
            </CollectionView>
        </Grid>
    </StackLayout>

</ContentPage>
```

App.xaml.cs | Database.cs | testTable.cs | AddSets.xaml | AddSets.xaml.cs

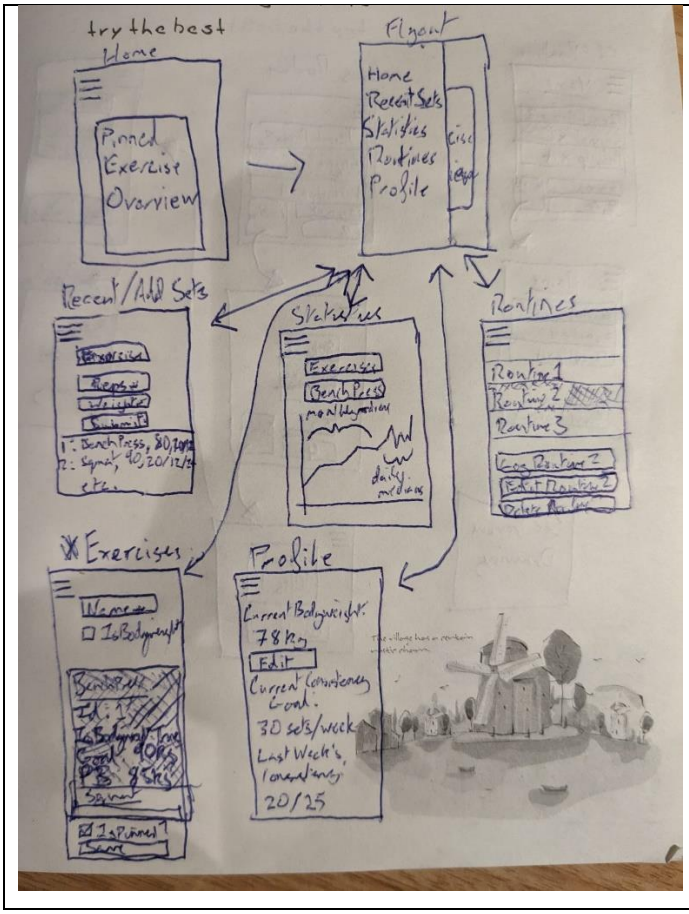
Fitness App | Fitness_App.AddSets | tempButton

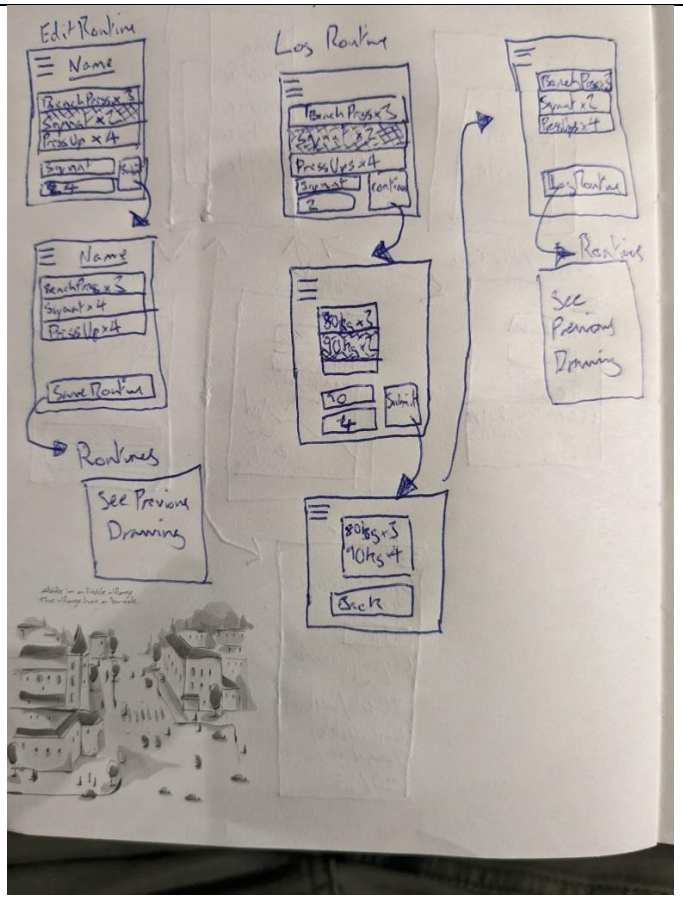
```
using System;
using System.Threading.Tasks;
using Xamarin.Forms;

namespace Fitness_App
{
    5 references
    public partial class AddSets : ContentPage
    {
        1 reference
        public AddSets()
        {
            InitializeComponent();
        }
        0 references
        protected override async void OnAppearing()
        {
            base.OnAppearing();
            testDataView.ItemsSource = await App.Database.GetTestDataAsync();
        }
        0 references
        private async void submitSet_Clicked(object sender, EventArgs e)
        {
            string xcize = xcizeInput.Text;
            int reps = Convert.ToInt16(repsInput.Text);
            float weight = Convert.ToSingle(weightInput.Text);
            DateTime date = DateTime.Now;
            System.Diagnostics.Debug.WriteLine("Clicked: xcize = {0}; reps = {1}; weight = {2}", xcize, reps, weight);
            if ((!string.IsNullOrEmpty(xcize)) && (reps > 0) && (weight > 0)) //sanitisation
            {
                await App.Database.SaveTestDataAsync(new testTable { XcizeAttribute = xcize,
                    RepsAttribute = reps, WeightAttribute = weight, DateAttribute = date }); // passes record obj into save method
                testDataView.ItemsSource = await App.Database.GetTestDataAsync();
                ((Button)sender).Text = "Submitted";
                await Task.Delay(2000);
                ((Button)sender).Text = "Submit Set";
            }
        }
        0 references
        private async void clearInputs_Clicked (object sender, EventArgs e)
        {
            xcizeInput.Text = repsInput.Text = weightInput.Text = "";
        }
        0 references
        private async void tempButton_Clicked (object sender, EventArgs e){
            // currently deletes oldest item in testTable
            await App.Database.DeleteTestDataAsync((await App.Database.GetTestItemAsync(1)));
            testDataView.ItemsSource = await App.Database.GetTestDataAsync();
        }
    }
}
```

Mock of UI

Drawings





Having settled on a Flyout navigation system, I drew up the above mock UI for my app.

Events

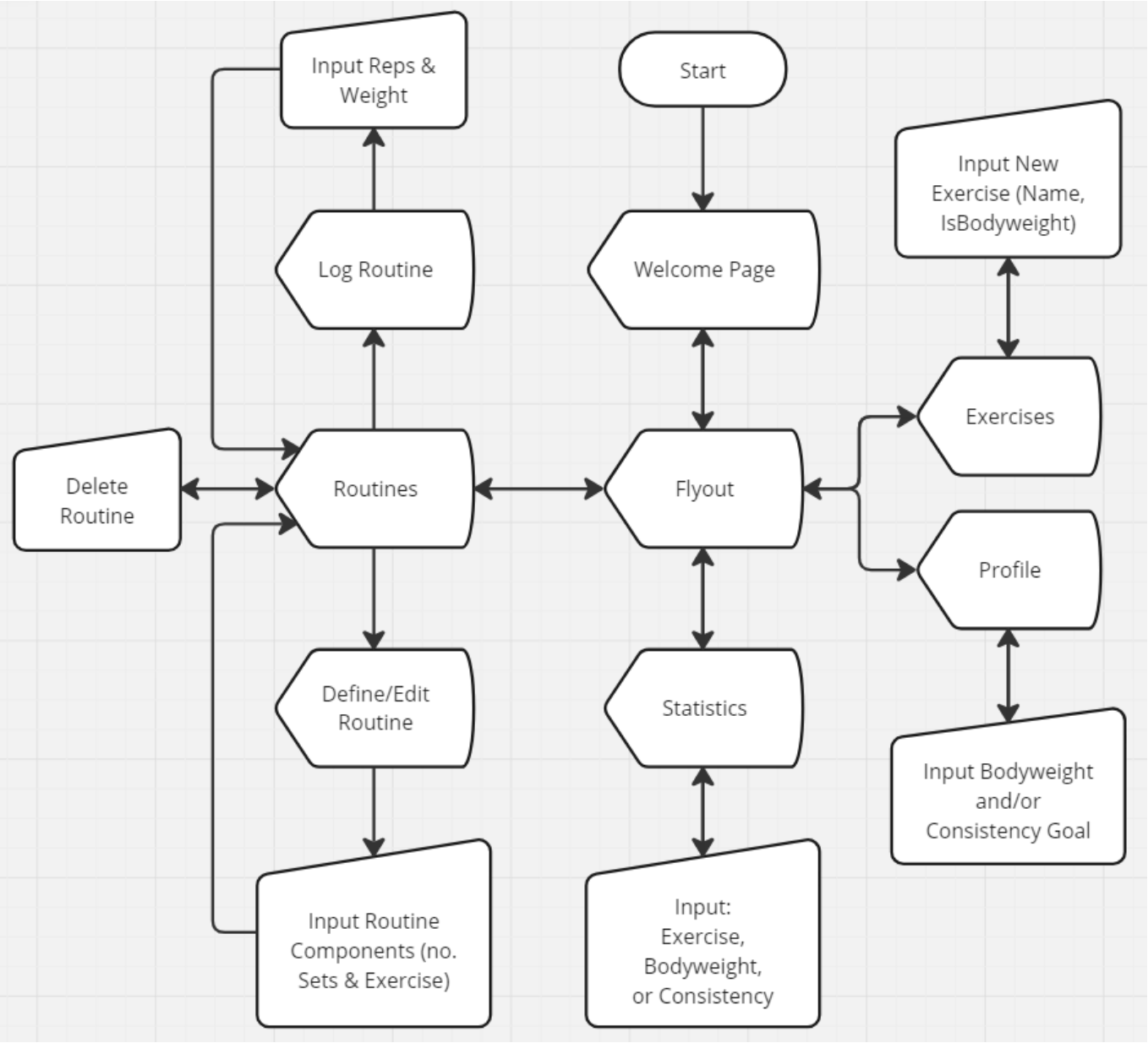
Here are the main events I'll need to deal with, having examined this mock of the UI.

- Page Loaded (all pages)
- Button Pressed (Add Sets, Exercises, Profile, Routines, Edit Routine, Log Routine)
- Selection Changed (Exercises, Routines, Edit Routine, Log Routine)
- Picker Changed (Statistics)

In my design section I will provide a full list of events and descriptions with how they are handled.

Documented Design

Top Level Flowchart



Events

Start Up

pp. 49-51 in Program Listing

- Initialise the Database
- Register Syncfusion License
- Redirect to Home Page

Home Page

pp. 76-78 in Program Listing

Load Home Page

- Pull pinned exercises from the Database
- Push pinned exercises to a table in the UI

Add Sets Page

pp. 78-82 in Program Listing

Load Add Sets Page

- Pull pending sets from the Database
- Push pending sets to the recent sets table in the UI
- Pull exercise names from the Database
- Push exercise names to the exercise picker in the UI

Submit Set Clicked

- Pull inputs from the exercise picker and the reps/weight entries in the UI
- Validate inputs
- If valid:
    - Push the inputted set to the Database

Undo Set Clicked

- Call the Database's Undo Set method
- Refresh the recent sets table in the UI

Redo Set Clicked

- Call the Database's Redo Sets method
- Refresh the recent sets table in the UI

Stats Page

pp. 82-87 in Program Listing

Load Stats Page

- Push options to the primary picker ('Exercises' or 'User Data')
- Pull the potential graph sources from the Database
- Load the two different possible sets of secondary options (For the option 'Exercises' -> A list of different Exercises; for the option 'User Data' -> the options 'Bodyweight' or 'Consistency')

Primary Picker Changed

- Pull primary picker input from the UI
- Push a different set of options to the secondary picker in the UI, depending on the option chosen in the primary picker
- Enable the secondary picker

Secondary Picker Changed

- Push the appropriate graph source to the graph in the UI, depending on the primary and secondary choices
- Set a binding path for the Y axis of the graph in the UI, depending on the primary and secondary choices

Routine Page

pp. 87-90 in Program Listing

Load Routine Page

- Pull routines from the Database
- Push routines to the routines table in the UI

Selected Routine Changed

- Pull the selected routine from the routines table in the UI
- Open a window showing different actions that could be taken on the selected routine (see below)

Delete Routine Clicked

Prerequisite: A routine must be selected

- Call the Database's Delete Routine Components method
- Call the Database's Delete Routine method
- Refresh the routines table in the UI
- Close the actions window

Edit Routine Clicked

Prerequisite: A routine must be selected

- Redirect to the Edit Routine Page with the ID of the selected routine as the argument

Log Routine Clicked

Prerequisite: A routine must be selected

- Redirect to the Log Routine Page with the ID of the selected routine as the argument

Define New Routine Clicked

- Redirect to Edit Routine Page with 0 as the argument

Edit Routine Page

pp. 89-94 in Program Listing

Load Edit Routine Page

- Store Routine ID parameter
- Pull exercise names from the Database
- Push exercise names to exercise picker in the UI
- If Routine ID parameter == 0:
    - Create an empty components list
- Else:
    - Pull routine name from the Database
    - Push routine name to routine name entry in the UI
    - Pull routine components from the Database, storing in a components list
- Push the components list to the components table in the UI

New Component Clicked

- Append a component to the components list
- Refresh the components table in the UI

Selected Component Changed

- Pull the selected component from the components table in the UI
- Open a window showing different actions that can be taken on the selected component (see below)

Save Component Clicked

Prerequisite: A component must be selected

- Pull inputs from the exercise picker and the sets entry in the UI
- Alter the selected component in the components list
- Refresh the components table in the UI
- Close the actions window

Delete Component Clicked

Prerequisite: A component must be selected

- Remove the selected component from the components list
- Refresh the components table in the UI
- Close the actions window

Save Routine Clicked

- Pull inputs from the routine name and sets entries as well as the exercise picker in the UI
- Validate inputs
- If valid:
    - Push the routine and its components to the Database
    - Redirect to Routines Page

Log Routine Page

pp. 95-104 in Program Listing

Load Log Routine Page

- Store Routine ID parameter
- Pull exercise names from the Database
- Push exercise names to exercise picker in the UI
- If Routine ID parameter == 0:
    - Create an empty components list
- Else:
    - Pull routine components from the Database, storing in a components list
- Push the components list to the components table in the UI

New Component Clicked

Prerequisite: Components table must be visible

- Append a component to the components list
- Refresh the components table in the UI

Selected Component Changed

Prerequisite: Components table must be visible

- Pull the selected component from the components table in the UI
- Open a window showing different actions that can be taken on the selected component (see below)

Delete Component Clicked

Prerequisite: A component must be selected

- Remove the selected component from the components list
- Refresh the components table in the UI
- Close the actions window

Define Component Clicked

Prerequisite: A component must be selected

- Pull inputs from the exercise picker and the sets entry in the UI
- Validate sets entry input
- If Valid:
    - Close the actions window
    - Hide the component table in the UI
    - Show the sets table in the UI
    - Alter the selected component in the components list
    - Create a list of set objects, as an attribute of the selected component, with length equal to the number of sets in that component.
    - Push set list to the sets table in the UI

Select Set Clicked

Prerequisite: Sets table must be visible

- Pull the selected set from the sets table in the UI
- Open an actions window allowing the user to save new set details (see below)

Save Set Clicked

Prerequisite: A set must be selected

- Pull inputs from the reps and weight entries in the UI
- Alter the selected set in the sets list
- Refresh the sets table in the UI
- Close the actions window

Finish Component Definition Clicked

Prerequisite: Sets table must be visible

- Close the actions window (if applicable)
- Hide the sets table in the UI
- Show the components table in the UI
- Refresh the components table in the UI

Log Workout Clicked

- Validate each component (and their associated sets) in the components list
- If valid:
    - Loop through components list
        - Loop through the sets list of that component, pushing each set to the Database
    - Redirect to Routines Page

Exercises Page

pp. 104-108 in Program Listing

Load Exercises Page

- Pull exercises from the Database
- Push exercises to the exercises table in the UI

Submit Exercise Clicked

- Pull inputs from the exercise name entry and the is bodyweight checkbox in the UI
- Validate inputs
- If valid:
    - Push exercise to the Database
    - Refresh the exercises table in the UI

Selected Exercise Changed

- Pull selected exercise from the exercises table in the UI
- Open an actions window allowing the user to pin the selected exercise (see below)

Pin Exercise Clicked

Prerequisite: An exercise must be selected

- Pull the input from the is pinned check box in the UI
- Push the new is pinned value for the selected exercise to the Database
- Refresh the exercises table in the UI

Profile Page

pp. 108-112 in Program Listing

Load Profile Page

- Pull user's data from the Database
- Push different elements of the data to the appropriate displays in the UI

Update Bodyweight Clicked

- Pull the input from the bodyweight input entry in the UI
- Validate the input
- If valid:
    - Push the user's updated bodyweight to the Database
    - Refresh the displays of the user's data in the UI

Update Consistency Goal Clicked

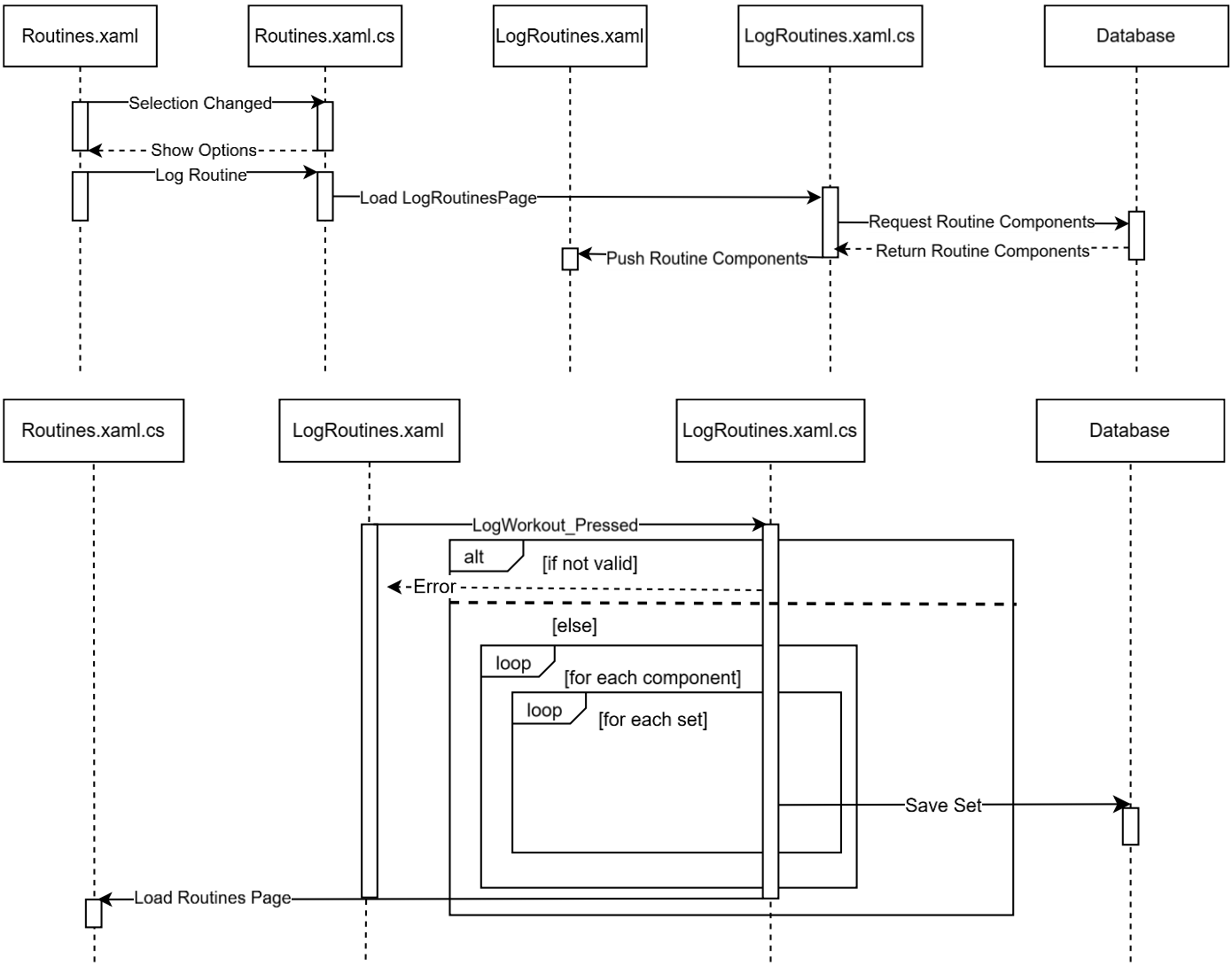
- Pull the input from the consistency goal entry in the UI
- Validate the input
- If valid:
    - Push new consistency goal to the Database
    - Refresh the displays of the user's data in the UI

Sequence Diagrams

pp. 88-90 & pp. 97-104 in Program Listing

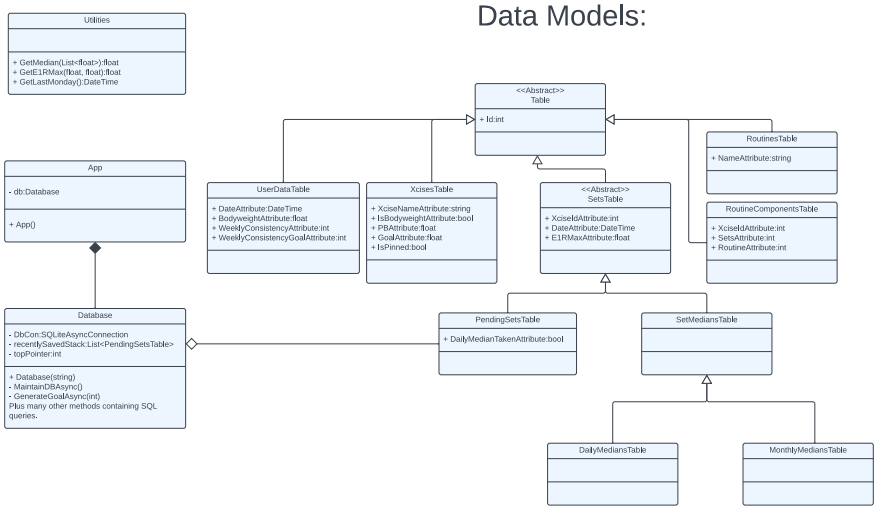
These are the sequence charts for the process of:

1) Opening the LogRoutinePage through the RoutinesPage and loading the routine data
2) Logging a workout through the LogRoutinePage

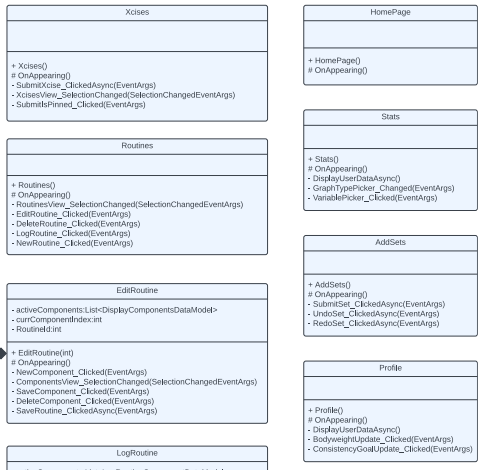


Class Diagram

Data Models:



Views:



Data Dictionaries

Xcises

Xcises is short for Exercises. This table will store details about each custom exercise that the user defines and the calculated values associated with it (PB and Goal).

Field Name	Data Type	Description	Validation	Key
Id	Integer	Unique identifier for Exercises.		PK
Name	String	Name of Exercise.	Length > 0	
IsBodyweight	Boolean	Indicates if bodyweight should be added to weight inputted for this exercise.		
PB	Float	The best e1RM recorded for this exercise.		
Goal	Float	The adaptive goal for e1RM for the next 3 months.	>= Last month's median	
IsPinned	Boolean	Indicates if the user has pinned this exercise's data to the home page.		

Pending Sets

This table stores recent sets submitted by the user that are being kept so that the daily and monthly medians for that day and month respectively can be taken.

Field Name	Data Type	Description	Validation	Key
Id	Integer	Unique identifier for Pending Sets.		PK
XciseId	Integer	Identifies the exercise type of the set.		FK
Date	DateTime	Date that the set was completed.		
E1RMax	Float	An estimate for the user's one rep max for this exercise based on this set only (by using a one rep max formula).		
DailyMedianTaken	Boolean	Indicates if a median for the day this set was recorded has been taken yet.		

Daily Medians

Daily Medians stores medians for each day this month that sets of a certain exercise were completed by the user.

Field Name	Data Type	Description	Validation	Key
Id	Integer	Unique identifier for Daily Medians.		PK
XciseId	Integer	Identifies the exercise type of the set.		FK
Date	DateTime	The day of the sets that the median was calculated for, in the format 12:00:00 d/m/y.		
E1RMax	Float	An estimate for the user's one rep max for this exercise based on sets on this day only (by taking a median).		

Monthly Medians

Monthly Medians stores medians for each month that sets of a certain exercise were completed by the user.

Field Name	Data Type	Description	Validation	Key
Id	Integer	Unique identifier for Monthly Medians.		PK
XciseId	Integer	Identifies the exercise type of the set.		FK
Date	DateTime	The month of the sets that the median was calculated for, in the format 12:00:00 1/m/y.		
E1RMax	Float	An estimate for the user's one rep max for this exercise based on sets on this day only (by taking a median).		

Routines

Routines stores the names of each routine. The identifier (Id) is used to group Routine Components together.

Field Name	Data Type	Description	Validation	Key
Id	Integer	Unique identifier for Routines.		PK
Name	String	Name of the workout routine.	Length > 0	

Routine Components

Routine Components make up Routines and is defined by a type of exercise and the number of sets of it that should be completed in a certain routine.

Field Name	Data Type	Description	Validation	Key
Id	Integer	Unique identifier for Routine Components.		PK
XciseId	Integer	Identifies the exercise type of the component.		FK
Sets	Integer	The number of sets of the specified exercise in this component of the workout.	> 0	
Routine	Integer	Identifies the workout routine that this component belongs to.		FK

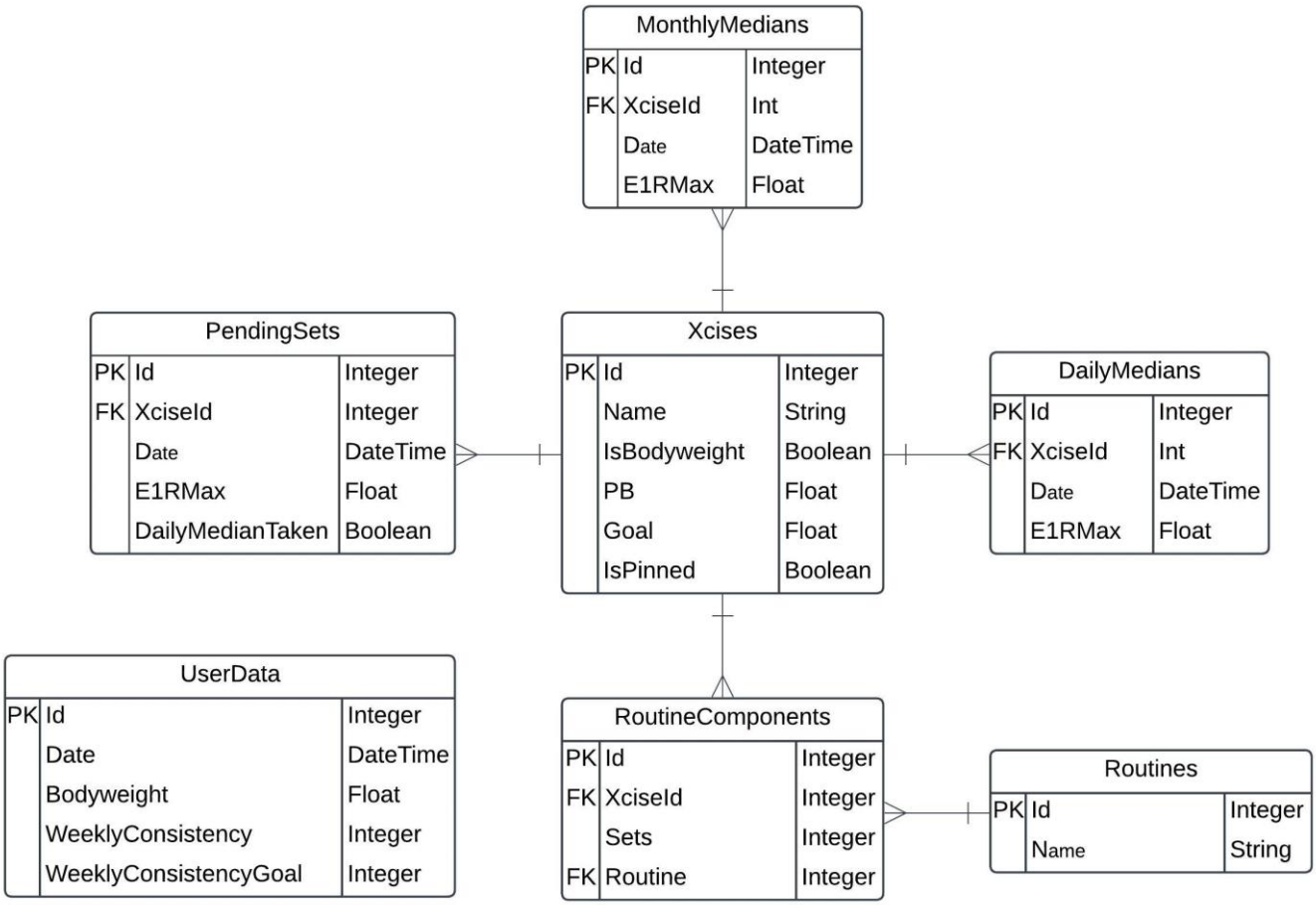
User Data

User Data stores information about the user each week they are active on the app. N.B. **this is a single user system**, so **all** this information **relates to one user**.

Field Name	Data Type	Description	Validation	Key
Id	Integer	Unique identifier for this weekly User Data log.		PK
Date	DateTime	The Monday of the week that the log refers to, in the format 12:00:00 d/m/y.		
Bodyweight	Float	The bodyweight of the user in this week.	> 0	
WeeklyConsistency	Integer	The total number of sets recorded in this week.		
WeeklyConsistencyGoal	Integer	The goal for WeeklyConsistency set in this week.	> 0	

Entity Relationship Diagram

pp. 72-76 in Program Listing



Data Structures

Undo/Redo Stack

p.44 & pp.48-59

This data structure is an adapted version of a stack that I came up with myself in order to satisfy objective 21:

21. User can undo/redo the adding of sets recorded in the current session

Whenever a user submits a set, the program will push that set to the undo/redo stack. If the user presses a button to undo the adding of the set, the undo/redo stack is popped, and the value's Id is used to delete that set from PendingSets.

The unique part is the redo function. If a user is unsure about their undoing of a set, they can press the redo button. This causes the program to look at the last item that was popped/undone (the value at the top pointer) and push it to the stack again.

A problem occurs when the user inputs the following:

- Adds set 1

	← TP
Set 1	

- Adds set 2

	← TP
Set 2	
Set 1	

- Undo

Set 2	← TP
Set 1	

- Undo

Set 2	
Set 1	← TP

- Adds set 3

Set 2	← TP
Set 3	

- Redo (e.g. pressed by accident)

Logically, you should only be able to redo a set if you have just undone one but would infact result in set 2 being added back in. To stop this, values at the top pointer and above should be wiped clean whenever a new set is added, which signifies 'a new branch' in the undo/redo tree (stopping the user from redoing until they do another undo).

Here are some example inputs to give an idea of how the undo/redo stack functions:

- Adds set 1

	← TP
Set 1	

- Adds set 2

	← TP
Set 2	
Set 1	

- Adds set 3

	← TP
Set 3	
Set 2	
Set 1	

- Adds set 4

	← TP
Set 4	
Set 3	
Set 2	
Set 1	

- Undo

Set 4	← TP
Set 3	
Set 2	
Set 1	

- Undo

Set 4	
Set 3	← TP
Set 2	
Set 1	

- Undo

Set 4	
Set 3	
Set 2	← TP
Set 1	

- Undo

Set 4	
Set 3	
Set 2	
Set 1	← TP

- Redo

Set 4	
Set 3	
Set 2	← TP
Set 1	

- Adds set 5

	← TP
Set 5	
Set 1	

- Adds set 6

	← TP
Set 6	
Set 5	
Set 1	

- Undo

Set 6	← TP
Set 5	
Set 1	

- Redo

	← TP
Set 6	
Set 5	
Set 1	

Selection Lists

pp. 87-89 (Routines), pp. 91-95 (EditRoutines), pp. 98-104 (LogRoutines) and pp. 106-108 (Exercises) in Program Listing

On the pages Exercises, Routines, Edit Routine and Log Routine, the user needs to be able to select an object from a list, so that it can be altered.

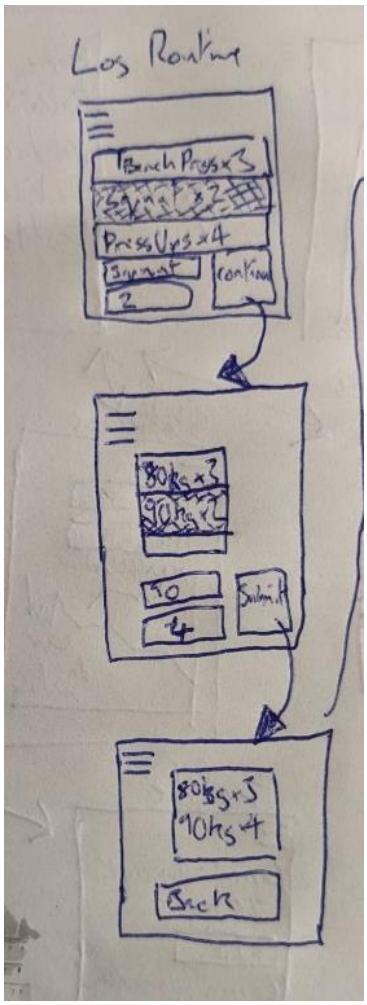
The way this is implemented is that there is a list of objects in the xaml.cs program which are pushed to a table in the UI. The table has an associated event called 'SelectionChanged', which sends a copy of the selected object to the xaml.cs program. The index of the selected object is then found in the list, and is recorded. This action then allows the user to input any changes to the object through the UI, which will be applied to the object in the list.

Log Routines Selection List

This particular list has another layer of complexity which is worth explaining, since the user needs to be able to change not only the exercise and number of sets in each component but also the number of reps and weight for each set in the decided number of sets.

Here's a demonstration of the desired effect, shown in my mock of the UI.

My commented pseudocode for the implementation of this can be found in the Log Routines Algorithms section under the Complex Algorithm Pseudocode heading.



Here's a reminder of what the LogRoutineComponent and SetLog Data Models look like in the class diagram with the relationships removed for simplicity.

LogRoutineComponentDataModel
+ Id:int + XciseIdAttribute:int + XciseNameAttribute:string + SetsAttribute:int + SetsList:List<SetLogDataModel>

SetLogDataModel
+ Index:int + Reps:float + Weight:float

As you can see, the log routine component data model (which the log routine selection list is comprised of) contains a further selection list in order for the user to select each set within the component and edit it.

Complex Algorithms Pseudocode

Maintain Database Algorithm

This is an algorithm to satisfy the objectives 6 a) and 6 b)

a) For each exercise, record the median e1RM for sets on the same day, on days this month.
b) For each exercise, record the median e1RM for sets in the same month, once the month has ended.

pp. 52-55 in Program Listing (split over multiple methods for cohesion)

This will run whenever the database class is booted up.

```
MaintainDBAsync()
    {
       DateTime startOfThisMonth = GetDateTime(1/Now.Month/Now.Year);
       data = GetPendingFromBeforeTodayAsync();
       xcisesList = GetXciseIdsAsync();
       foreach (xcise in xcisesList)
       {
         xciseData = data.Where(x.XciseIdAttribute == xcise);

          // Storing Daily Median
          dailyData = xciseData.Where(x.DailyMedianTaken== false);
          if (dailyData.Count() != 0)
          {
             UpdateXcisePB(dailyData, xcise);
             daysList = getUniqueDays(dailyData);
             foreach (day in daysList)
             {
                E1RMaxes = dailyData.Where(x.DateAttribute == day).Select(x.E1RMax);
                e1RMaxMedian = GetMedian(E1RMaxes);
                Db.InsertAsync(new DailyMediansTable
                {
                   XciseId = xcise,
                   Date = day,
```

```
                E1RMax = e1RMaxMedian
            });
        }
    }

    // Storing Monthly Median
    monthlyData = xciseData.Where(x.DateAttribute < startOfThisMonth);
    if (monthlyData.Count() != 0)
    {
        monthsList = getUniqueMonths(monthlyData)
        foreach (month in monthsList)
        {
            LB = month;
            UB = LB.AddMonths(1);
            E1RMaxes = monthlyData.Where(x.Date >= LB && x.Date < UB).Select(x.E1RMax);
            e1RMaxMedian = GetMedian(E1RMaxes);
            Db.InsertAsync(new MonthlyMediansTable
            {
                XciseIdAttribute = xcise,
                DateAttribute = month,
                E1RMaxAttribute = e1RMaxMedian
            });
        }
    }
    SetRecordDailyMedianTakenForPendingBeforeTodayAsync()
    DeletePendingFromLastMonthAsync()
    DeleteDailyMediansFromLastMonthAsync()
}
```

Generate Goals Algorithm

This is the algorithm to generate goals, to satisfy objective 13:

13) App can generate an adaptive goal specific to the user.

pp. 55-56 in Program Listing

This will be run whenever a new monthly median is calculated.

```
GenerateGoal(Xcise)
    {
        const NumPoints = 3; // how many months the regression backdates uses
        const GoalLength = 3; // how many months ahead the goal is predicting (3 means in 3 calendar months time)
        monthMedians = GetXciseMonthlyMediansAsync(Xcise);
        if (monthMedians.Count() >= NumPoints)
        {
            datapoints = new List {};
            for (i = 1 - NumPoints; i <= 0; i++)
            {
                datapoints.Add((i, monthMedians[monthMedians.Count() - 1 + i].E1RMax));
            } // adds the last n monthMedians to datapoints

            tSum = datapoints.Select(x.t).Sum();
            t2Sum = getSumOfSquaresOfT(datapoints);
            e1RMSum = datapoints.Select(x.e1RM).Sum();
            te1RMSum = datapoints.Select(x.t * x.e1RM).Sum();

            gradient = ((NumPoints * te1RMSum) - (tSum * e1RMSum)) / ((NumPoints * t2Sum) – square(tSum)); // regression formula
            // linear regression formula: gradient = (n∑xy - ∑x*∑y) / (n∑x^2 - (∑x)^2)
            if (gradient > 0) { goal = datapoints.Last().e1RM + gradient * (GoalLength+1); }
            else { goal = datapoints.Last().e1RM; }  // If goal is negative, no point giving goal less than current 1RMax

            await Db.Query("UPDATE XcisesTable SET Goal = ? WHERE Id = ?;", goal, Xcise);
        }
    }
```

Log Routines Page Algorithms

This set of algorithms, contained within the class for the LogRoutine page, work together to implement a two dimensional selection list to facilitate objective 15:

15) User can log user-defined workout routines quickly, by loading up a workout then inputting, for each set, the number of reps and the weight.

The pseudocode for the checking and submission of the sets is not included.

pp. 97-102 in Program Listing

This class will contain all the methods of the LogRoutine page that are relevant to this process.

```
class LogRoutine:
  {
    Initialise activeComponents; // list of the components that are being shown in the UI
    Initialise currComponentIndex; // index of the current selection
    Initialise currSetIndex; // each LogRoutineComponentDataModel has a list of sets, this refers to the index of the current selection in that list.
    Initialise readonly RoutineId;
    LogRoutine(inputRoutineId) // takes the routine needing to be logged as an argument
    {
      InitializeComponent();
      RoutineId = inputRoutineId;
      xcisePicker.ItemsSource = App.Db.GetXciseNamesAsync();
      activeComponents = App.Db.GetRoutineComponentsToLogAsync(RoutineId);
      ComponentsView = activeComponents
    NewComponent_Clicked(event)
    {
      if (activeComponents.Count() == 0)
      {
        activeComponents.Add(new LogRoutineComponentDataModel() { Id = 1 });   // assigns a temporary ID of 1, if list is empty
      } else
      {
        activeComponents.Add(new LogRoutineComponentDataModel() { Id = activeComponents.Last().Id + 1 });   // assigns an incremented temp ID
      }
```

```
        ComponentsView.Reload()
    }
    ComponentsView_SelectionChanged(event)
    {
        currComponentIndex = activeComponents.IndexOf(event.CurrentSelection); // assigns currComponentIndex to selected
        ComponentEdits.IsVisible = true; // shows buttons to allow interaction with curr. component
        xcisePicker.SelectedIndex = activeComponents[currComponentIndex].XciseId- 1; // assigns picker to current value of exercise
        setsInput.Text = str(activeComponents[currComponentIndex].Sets); // assigns sets field to current value of sets
    }

    DeleteComponent_Clicked(event)
    {
        activeComponents.RemoveAt(currComponentIndex);
        ComponentEdits.IsVisible = false; // close edits window
        ComponentsView.Reload()
    }
    DefineComponent_Clicked(event)
    {
        if (int(setsInput.Text) > 0) // sanitation
        {
            setsChanged = false;
            AlterRoutine.IsVisible = false; // closes window for selecting a routine component
            SetsData.IsVisible = true; // opens window for selecting sets within the selected routine component
            if (activeComponents[currComponentIndex].Sets != int(setsInput.Text)) // need to regenerate SetsList
            {
                setsChanged = true;
                activeComponents[currComponentIndex].Sets = int(setsInput.Text);
            }
```

```
            activeComponents[currComponentIndex].XciseId = xcisePicker.SelectedIndex + 1; // exercise picker is loaded in order of id, so index + 1 = exercises id
            activeComponents[currComponentIndex].XciseName = xcisePicker.SelectedItem.XciseName;
            if (activeComponents[currComponentIndex].SetsList == null || setsChanged == true)
            {
               expectedInputs = new List();
               for (i = 0; i < activeComponents[currComponentIndex].Sets; i++) // constructs list of sets to be edited
               {
                  expectedInputs.Add(new SetLogDataModel { Index = i });
               }
               activeComponents[currComponentIndex].SetsList = expectedInputs;
            }
            SetsView.ItemsSource = activeComponents[currComponentIndex].SetsList;
         }
      }

      SetsView_SelectionChanged (event)
      {
         currSetIndex = event.CurrentSelection.Index;
         SetEdits.IsVisible = true; // opens sets editing window
         repsInput.Text = str(activeComponents[currComponentIndex].SetsList[currSetIndex].Reps); // assigns current value of reps to the reps field
            weightInput.Text = str(activeComponents[currComponentIndex].SetsList[currSetIndex].Weight); // assigns current value of weight to the weight field.
      }

      SaveSet_Clicked(events)
      {
         if (isEmpty(repsInput.Text)) { reps = 0; } else { reps = float(repsInput.Text); };
         if (isEmpty(weightInput.Text)) { weight = 0; } else { weight = float(weightInput.Text); };
         activeComponents[currComponentIndex].SetsList[currSetIndex].Reps = reps;
```

```
    activeComponents[currComponentIndex].SetsList[currSetIndex].Weight = weight;
    SetsView.Reload();
    SetEdits.IsVisible = false; // closes editing sets window
}
EditFinished_Clicked(events)
{
    SetsData.IsVisible = false; // closes window for viewing sets
    AlterRoutine.IsVisible = true; // opens window for viewing routine components
    ComponentsView.Reload()
}
```

SQL Queries

pp. 52-68 in Program Listing

Create Queries

INSERT INTO PendingSets (XciseId, Date, E1RMax, DailyMedianTaken) VALUES w, x, y, z;

Records sets / using the redo set function.

INSERT INTO DailyMedians (XciseId, Date, E1RMax) VALUES x, y, z;

Stores a daily median.

INSERT INTO MonthlyMedians (XciseId, Date, E1RMax) VALUES x, y, z;

Stores a monthly median.

INSERT INTO Xcises (Name, IsBodyweight) VALUES x, y;

Defines a new exercise.

INSERT INTO Routines (Name) VALUES x;

Defines a new routine.

INSERT INTO RoutineComponents (XciseId, Sets, Routine) VALUES x, y, z;

Adds a new component to an existing routine.

INSERT INTO UserData (Bodyweight, Date, WeeklyConsistency, WeeklyConsistencyGoal) VALUES w, x, y, z;

Creates a new record of the user's personal data for the current week.

Read Queries

SELECT * FROM PendingSetsTable WHERE Date <> x ORDER BY E1RMax DESC;

Let x be today's date.

Gets pending data so that the appropriate medians can be taken and recorded.

SELECT PendingSetsTable.*, XcisesTable.XciseName FROM PendingSetsTable

INNER JOIN XcisesTable ON PendingSetsTable.XciseId = XcisesTable.Id

ORDER BY DateAttribute DESC, Id DESC;

Gets recent sets from the table pending sets so that they can be displayed in the UI (XciseId needs to be replaced with XciseName)

SELECT Id FROM XcisesTable;

Gets a list of valid exercise Ids to iterate through.

SELECT Id, XciseName FROM XcisesTable;

Gets a list of exercise names so that the user can select an exercise using a picker.

SELECT * FROM XcisesTable WHERE IsPinned = TRUE;

Gets all the data associated with each exercise that has been pinned to the homepage by the user.

```
SELECT Name FROM RoutinesTable WHERE Id = x;
```

For getting the name of a certain routine.

```
SELECT RoutineComponentsTable.XciseId, RoutineComponentsTable.Sets, XcisesTable.XciseName
FROM RoutineComponentsTable
INNER JOIN XcisesTable ON RoutineComponentsTable.XciseId = XcisesTable.Id
WHERE Routine = x;
```

Gets all the components of a routine so that they can be displayed in the UI (XciseId needs to be replaced with XciseName).

```
SELECT * FROM DailyMediansTable UNION SELECT * FROM MonthlyMediansTable ORDER BY DateAttribute ASC;
```

Gets the data for daily and monthly medians so they can be later sorted by exercise type and displayed on a graph.

```
SELECT E1RMax, Date FROM MonthlyMediansTable WHERE XciseId = x ORDER BY Date ASC;
```

Gets a list of monthly medians for a certain exercise, so a goal can be generated based on a regression calculator.

```
SELECT Bodyweight, Date FROM UserDataTable;
```

Gets a list containing the user's recorded bodyweight over time, so that it can be displayed on a graph.

```
SELECT Consistency, Date FROM UserDataTable WHERE Date < x;
```

Let x be the Monday of the current week.

Gets a list containing the user's recorded weekly consistency over time (excluding the current week), so that it can be displayed on a graph.

```
SELECT IsBodyweight FROM XcisesTable WHERE Id = x;
```

Determines if a certain exercise is a bodyweight exercise.

```
SELECT * FROM UserData ORDER BY Date DESC;
```

Gets a list containing the user's historic data, which can be used to get the most up to date data.

```
SELECT PB FROM Xcises WHERE Id = x;
```

Gets the e1RMax for the user's personal best set stored for a certain exercise.

Update Queries

```
UPDATE PendingSets SET DailyMedianTaken = TRUE
WHERE ((DailyMedianTaken = FALSE) AND (Date <> x));
```

Let x be today's date.

Indicates which sets have had their daily median taken (all sets in PendingSets except those recorded today). This runs at the end of the MaintainDBAsync() function on start up.

UPDATE Xcises SET Goal = x WHERE Id = y;

Sets a new goal for a certain exercise.

UPDATE Xcises SET IsPinned = x WHERE Id = y;

Sets a certain exercise as pinned on the homepage.

UPDATE Routines SET XciseName = x WHERE Id = y;

Changes the name of a certain routine.

UPDATE UserData SET Bodyweight = x WHERE Date = y;

Let y be the date of this week's Monday.

Updates the user's bodyweight.

UPDATE UserData SET WeeklyConsistency = x WHERE Date = y;

Let y be the date of this week's Monday.

Updates the user's weekly consistency score for this week.

UPDATE Xcises SET PB = ? WHERE Id = ?;

Updates the e1RMax for the user's personal best set for a certain exercise.

Delete Queries

DELETE FROM PendingSets WHERE Date < x;

Let x be the date of the start of this month.

Deletes pending sets older than a month (unnecessary since both daily and monthly medians have been taken).

DELETE FROM DailyMedians WHERE Date < x;

Let x be the date of the start of this month.

Deletes daily medians older than a month (they are unnecessary since monthly medians have been taken).

DELETE FROM PendingSetsTable WHERE Id = x ;

Deletes a certain pending set (for the undo function).

DELETE FROM RoutinesTable WHERE Id = x;

Deletes a certain routine.

DELETE FROM RoutineComponentsTable WHERE RoutineAttribute = ?;

Deletes all components from a certain routine (either for deleting a routine, or for changing it in which case new components will be inserted).

Technical Solution

In my program listing, I will highlight and footnote areas (see below for a summary) in which I feel I have demonstrated a particular high-level skill (based off my interpretation of the AQA NEA specification).

Because I am footnoting areas I have showed certain skills, the program wont work if simply copy and pasted.

Data Models
Data Structures
Defensive Programming
Self-Defined use of OOP
Self-Defined Algorithms
Exception Handling
Dynamic Generation of Objects
Cross-Table SQL Queries
Mathematical Models
Use of Constants / Read-Only Objects

-N.B. I am not noting everything which comes under these categories, only parts that are particularly complex. Furthermore, there are coding styles that I feel that I have used that are not in this table as they are too broad to go through and highlight in my program listing.

In my program, I have followed the following conventions

- Classes, Methods and Attributes Capitalised
- Descriptive naming, in Title or Camel Case
- No use of global variables
- Methods and Classes are cohesive
- Comments in more convoluted sections of the code

I have omitted all program files which entirely came from the template and have not been altered, other parts of the template are clearly labelled in the program listing using comments. For this reason, the navigation system is not described in the following program listing (see my prototyping section for further details).

Program Listing

App.xaml

```
<?xml version="1.0" encoding="utf-8" ?>
<Application xmlns="http://xamarin.com/schemas/2014/forms"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="Fitness_Appv2.App">
    <!-- **** TEMPLATE CODE START **** -->
    <!--
        Define global resources and styles here, that apply to all pages in your app.
    -->
    <Application.Resources>
        <ResourceDictionary>

            <Color x:Key="Primary">#2196F3</Color>
            <Style TargetType="Button">
                <Setter Property="TextColor" Value="White"></Setter>
                <Setter Property="VisualStateManager.VisualStateGroups">
                    <VisualStateGroupList>
                        <VisualStateGroup x:Name="CommonStates">
                            <VisualState x:Name="Normal">
                                <VisualState.Setters>
                                    <Setter Property="BackgroundColor" Value="{StaticResource Primary}" />
                                </VisualState.Setters>
                            </VisualState>
                            <VisualState x:Name="Disabled">
                                <VisualState.Setters>
                                    <Setter Property="BackgroundColor" Value="#332196F3" />
                                </VisualState.Setters>
                            </VisualState>
                        </VisualStateGroup>
                    </VisualStateGroupList>
```

```
            </Setter>
        </Style>
    <!-- **** TEMPLATE CODE END **** -->

        <Style TargetType="Label">
            <Setter Property="TextColor" Value="DarkGray"/>
        </Style>
        <Style TargetType="Entry">
            <Setter Property="TextColor" Value="Black"/>
            <Setter Property="PlaceholderColor" Value="DarkGray"/>
        </Style>
        <Style TargetType="Picker">
             <Setter Property="TextColor" Value="Black"/>
             <Setter Property="TitleColor" Value="DarkGray"/>
        </Style>
    </ResourceDictionary>
</Application.Resources>
</Application>
```

App.xaml.cs

```
using Fitness_Appv2.Services;
using System;
using Xamarin.Forms;
using System.IO;
namespace Fitness_Appv2
{
    public partial class App : Application
    {
        private static Database db; // Initiates new Db object 'db' as a property
        public static Database Db
        {
            get
```

```
            {
                if (db == null)
                {
                    db = new Database(Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.LocalApplicationData),
"FitnessAppv2.db3")); // creates db on disk
                }
                return db;
            } // Getter for 'db' object
        }

        public App()
        {
            Syncfusion.Licensing.SyncfusionLicenseProvider.RegisterLicense("Ngo9BigBOggjHTQxAR8/V1NMaF5cXmBCf1FpRmJGdld5fUVHYVZUTXxaS00DNHVRd
kdmWX5fcXVVRWFcUUNyXko=");
            // **** TEMPLATE CODE START ****
            InitializeComponent();
            MainPage = new AppShell();
            // **** TEMPLATE CODE END ****
        }
    }
}
```

Services Folder

Database.cs

```
using SQLite;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

namespace Fitness_Appv2.Services
{
```

```
    public class Database
    {
        private readonly SQLiteAsyncConnection DbCon; // db connection object initialised
        private readonly List<PendingSetsTable> undoRedoStack = new List<PendingSetsTable>() { null };[1] [2] // top of stack should always be a
null
        private int topPointer = 0; // points to next free index
        public Database(string DbPath) // Db class constructor
        {
            DbCon = new SQLiteAsyncConnection(DbPath); // db connection assigned, using location of db on disk as argument
            DbCon.CreateTableAsync<PendingSetsTable>(); // Creates PendingSetsTable on startup
            DbCon.CreateTableAsync<DailyMediansTable>();
            DbCon.CreateTableAsync<MonthlyMediansTable>();
            DbCon.CreateTableAsync<XcisesTable>();
            DbCon.CreateTableAsync<UserDataTable>();
            DbCon.CreateTableAsync<RoutineComponentsTable>();
            DbCon.CreateTableAsync<RoutinesTable>();
            MaintainDBAsync();
        }
        private async void MaintainDBAsync()[3]
        {
            await CheckNeedNewUserDataAsync();


            List<PendingSetsTable> data = await DbCon.QueryAsync<PendingSetsTable>("SELECT * FROM PendingSetsTable WHERE DateAttribute <> ?
ORDER BY E1RMaxAttribute DESC;", DateTime.Today);
            if (data.Count() != 0)
            {
                UpdateWeeklyConsistencyAsync(data.Where(obj => obj.DailyMedianTakenAttribute == false).ToList());
```

[1] Data Structures
[2] Use of Read-Only Objects
[3] Self-Defined Algorithms

```
            }

            DateTime startOfThisMonth = new DateTime(DateTime.Today.Year, DateTime.Today.Month, 1);
            List<int> xcisesList = (await GetXciseIdsAsync()).Select(obj => obj.Id).ToList();
            foreach (int xcise in xcisesList)
            {
                List<PendingSetsTable> PBData = data.Where(obj => obj.XciseIdAttribute == xcise).ToList();

                if (PBData.Count() != 0)
                {
                    UpdateXcisePBAsync(xcise, PBData[0].E1RMaxAttribute);
                }

                TakeMediansAsync(xcise, data, startOfThisMonth);
            }

            await DbCon.QueryAsync<PendingSetsTable>("UPDATE PendingSetsTable SET DailyMedianTakenAttribute = TRUE WHERE
((DailyMedianTakenAttribute = FALSE) AND (DateAttribute <> ?));", DateTime.Today);
            await DbCon.QueryAsync<PendingSetsTable>("DELETE FROM PendingSetsTable WHERE DateAttribute < ?;", startOfThisMonth);
            await DbCon.QueryAsync<DailyMediansTable>("DELETE FROM DailyMediansTable WHERE DateAttribute < ?;", startOfThisMonth);
        }

        public void TakeMedians(int xcise, List<PendingSetsTable> data, DateTime startOfThisMonth)[4]
        {
            List<PendingSetsTable> xciseData = data.Where(obj => obj.XciseIdAttribute == xcise).ToList();

            List<PendingSetsTable> dailyData = xciseData.Where(obj => obj.DailyMedianTakenAttribute == false).ToList();
            if (dailyData.Count() != 0)
            {
                TakeDailyMediansAsync(xcise, dailyData);
```

[4] Self-Defined Algorithms

```
            }

            List<PendingSetsTable> monthlyData = xciseData.Where(obj => obj.DateAttribute < startOfThisMonth).ToList();
            if (monthlyData.Count() != 0)
            {
                TakeMonthlyMediansAsync(xcise, monthlyData);
            }
        }
        public async void TakeDailyMediansAsync(int xcise, List<PendingSetsTable> dailyData)
        {
            List<DateTime> daysList = dailyData.Select(obj => obj.DateAttribute).Distinct().ToList();
            foreach (DateTime day in daysList)
            {
                List<float> E1RMaxes = dailyData.Where(obj => obj.DateAttribute == day).Select(obj => obj.E1RMaxAttribute).ToList();
                float e1RMaxMedian = Utilities.GetMedian(E1RMaxes);

                await DbCon.InsertAsync(new DailyMediansTable
                {
                    XciseIdAttribute = xcise,
                    DateAttribute = day,
                    E1RMaxAttribute = e1RMaxMedian
                });
            }
        }
        public async void TakeMonthlyMediansAsync(int xcise, List<PendingSetsTable> monthlyData)
        {
            List<DateTime> monthsList = monthlyData.Select(obj => new DateTime(obj.DateAttribute.Year, obj.DateAttribute.Month,
1)).Distinct().ToList();
            foreach (DateTime month in monthsList)
```

[5] Self-Defined Algorithms
[6] Self-Defined Algorithms

```
            {
                DateTime LB = month;
                DateTime UB = LB.AddMonths(1);
                List<float> E1RMaxes = monthlyData.Where(obj => obj.DateAttribute >= LB && obj.DateAttribute < UB).Select(obj =>
obj.E1RMaxAttribute).ToList();
                float e1RMaxMedian = Utilities.GetMedian(E1RMaxes);
                await DbCon.InsertAsync(new MonthlyMediansTable
                {
                    XciseIdAttribute = xcise,
                    DateAttribute = month,
                    E1RMaxAttribute = e1RMaxMedian
                });
                GenerateGoalAsync(xcise); // Latest Monthly Median Changing --> new Goals
            }
        }


        public async void GenerateGoalAsync(int Xcise)[7]
        {
            const int NumPoints = 3; // how many months the regression backdates uses
            const int GoalLength = 3;[8] // how many months ahead the goal is predicting (3 means in 3 +- 0.5 months depending on the day)
            List<SetMediansTable> monthMedians = await GetXciseMonthlyMediansAsync(Xcise);
            if (monthMedians.Count() >= NumPoints)
            {
                List<(int t, float e1RM)> datapoints = new List<(int, float)> { };
                for (int i = 1 - NumPoints; i <= 0; i++)
                {
                    datapoints.Add((i, monthMedians[monthMedians.Count() - 1 + i].E1RMaxAttribute)); // for NumPoints = 3: i is -2, -1, 0
                } // adds the last n monthMedians to datapoints
```

[7] Self-Defined Algorithms
[8] Use of constants

```
                int tSum = datapoints.Select(x => x.t).Sum();
                int t2Sum = Convert.ToInt16(datapoints.Select(x => Math.Pow(x.t, 2)).Sum());
                float e1RMSum = datapoints.Select(x => x.e1RM).Sum();
                float te1RMSum = datapoints.Select(x => x.t * x.e1RM).Sum();

                float gradient = ((NumPoints * te1RMSum) - (tSum * e1RMSum)) / ((NumPoints * t2Sum) - Convert.ToInt16(Math.Pow(tSum, 2)));[9]
// regression formula
                // linear regression formula: gradient = (n∑xy - ∑x*∑y) / (n∑x^2 - (∑x)^2)
                float goal;
                if (gradient > 0) { goal = datapoints.Last().e1RM + gradient * (GoalLength + 1); }
                else { goal = datapoints.Last().e1RM; }[10] [11]// Will not give a goal that is less than the last month

                await DbCon.QueryAsync<XcisesTable>("UPDATE XcisesTable SET GoalAttribute = ? WHERE Id = ?;", goal, Xcise);
            }
        }

        // **** PendingSetsTable Methods Start ****
            public async Task<List<DisplaySetsDataModel>> GetPendingSetsToViewAsync()
            {
                try
                {
                    return await DbCon.QueryAsync<DisplaySetsDataModel>(@"SELECT PendingSetsTable.*, XcisesTable.XciseNameAttribute
                                                                          FROM PendingSetsTable
                                                                         INNER JOIN XcisesTable
                                                                          ON PendingSetsTable.XciseIdAttribute = XcisesTable.Id
                                                                         ORDER BY DateAttribute DESC, Id DESC;");[12] // sorts by Id within
the same date
```

[9] Mathematical Models
[10] Defensive Programming
[11] Mathematical Models
[12] Cross-Table SQL Queries

```
                }
                catch
                {
                    return null;
                }[13]
            }
            public async Task SaveSetAsync(PendingSetsTable saveData)
            {
                undoRedoStack[topPointer] = saveData;
                topPointer++;
                if (topPointer == undoRedoStack.Count()) { undoRedoStack.Add(null); } // if at top of stack add null value (makes checking redo validity simpler)
                for (int i = topPointer; i < undoRedoStack.Count(); i++) { undoRedoStack[i] = null; }[14] // clears anything above topPointer as new branch has started so redo should be impossible

               await DbCon.InsertAsync(saveData);
                // when this happens the respective object in the undoRedoStack has its Id updated (autoincrement has a delayed effect?)
            }

            public async void UndoSetAsync()
            {
                if (topPointer != 0)
                {
                    topPointer--;[15]
                    await DbCon.QueryAsync<PendingSetsTable>("DELETE FROM PendingSetsTable WHERE Id = ?;", undoRedoStack[topPointer].Id);
                }
            }
            public async void RedoSetAsync()
```

[13] Exception Handling
[14] Data Structures
[15] Data Structures

```
            {
                if (undoRedoStack[topPointer] != null)
                {
                    await DbCon.InsertAsync(undoRedoStack[topPointer]);

                    topPointer++;
                    if (topPointer == undoRedoStack.Count()) { undoRedoStack.Add(null); }[16]
                }
            }
        // **** PendingSetsTable Methods End ****

        // **** XcisesTable Methods Start ****
            public async Task<List<XcisesTable>> GetXcisesAsync()
            {
                try
                {
                    return await DbCon.Table<XcisesTable>().ToListAsync();
                }
                catch
                {
                    return null;
                }[17]
            }

            public async Task SaveXciseAsync(XcisesTable saveData)
            {
               await DbCon.InsertAsync(saveData);
            }
```

[16] Data Structures
[17] Exception Handling

```
        public async Task<List<XcisesTable>> GetXciseIdsAsync()
        {
            try
            {
                return (await DbCon.QueryAsync<XcisesTable>("SELECT Id FROM XcisesTable;")).ToList();
            }
            catch
            {
                return null;
            }[18]
        }

        public async Task<List<XcisesTable>> GetXciseNamesAsync()
        {
            try
            {
                return (await DbCon.QueryAsync<XcisesTable>("SELECT Id, XciseNameAttribute FROM XcisesTable;"));
            }
            catch
            {
                return null;
            }[19]
        }

        public async void UpdateXciseIsPinnedAsync(Boolean newIsPinned, int id)
        {
            await DbCon.QueryAsync<XcisesTable>("UPDATE XcisesTable SET IsPinnedAttribute = ? WHERE Id = ?;", newIsPinned, id);
        }
```

[18] Exception Handling
[19] Exception Handling

```
        public async Task<List<XcisesTable>> GetPinnedXcisesAsync()
        {
            return await DbCon.QueryAsync<XcisesTable>("SELECT * FROM XcisesTable WHERE IsPinnedAttribute = TRUE;");
        }

    // **** XcisesTable Methods End ****

    // **** RoutinesTable Methods Start ****
        public async Task<List<RoutinesTable>> GetRoutinesAsync()
        {
            try
            {
                return await DbCon.Table<RoutinesTable>().ToListAsync();
            }
            catch
            {
                return null;
            }[20]
        }

        public async Task<int> SaveRoutineAsync(RoutinesTable saveData) // returns id of new routine

        {
            await DbCon.InsertAsync(saveData);
            return saveData.Id;
        }
        public async Task<string> GetRoutineNameAsync(int id)
        {
            try
            {
```

[20] Exception Handling

```
                    return (await DbCon.QueryAsync<RoutinesTable>("SELECT NameAttribute FROM RoutinesTable WHERE Id = ?;",
id))[0].NameAttribute;
                }
                catch
                {
                    return null;
                }[21]
            }

            public async void UpdateRoutineNameAsync(string name, int id)
            {
                await DbCon.QueryAsync<RoutinesTable>("UPDATE RoutinesTable SET XciseNameAttribute = ? WHERE Id = ?;", name, id);
            }

            public async void DeleteRoutineAsync(int id)
            {
                await DbCon.QueryAsync<RoutinesTable>("DELETE FROM RoutinesTable WHERE Id = ?;", id);
            }

        // **** RoutinesTable Methods End ****

        // **** RoutineComponentsTable Methods Start ****
            public async void SaveRoutineComponentAsync(RoutineComponentsTable saveData)
            {
                await DbCon.InsertAsync(saveData);
            }

            public async Task<List<LogRoutineComponentDataModel>> GetRoutineComponentsToLogAsync(int routineId)
            {
```

[21] Exception Handling

```
                try
                {
                    return (await DbCon.QueryAsync<LogRoutineComponentDataModel>(@"
SELECT RoutineComponentsTable.XciseIdAttribute, RoutineComponentsTable.SetsAttribute, XcisesTable.XciseNameAttribute
FROM RoutineComponentsTable
INNER JOIN XcisesTable ON RoutineComponentsTable.XciseIdAttribute = XcisesTable.Id
WHERE RoutineAttribute = ?;", routineId)).ToList();[22]
                }
                catch
                {
                    return null;
                }[23]
            }

            public async Task<List<DisplayComponentsDataModel>> GetRoutineComponentsToEditAsync(int routineId)
            {
                try
                {
                    return (await DbCon.QueryAsync<DisplayComponentsDataModel>(@"
SELECT RoutineComponentsTable.XciseIdAttribute, RoutineComponentsTable.SetsAttribute, XcisesTable.XciseNameAttribute
FROM RoutineComponentsTable
INNER JOIN XcisesTable ON RoutineComponentsTable.XciseIdAttribute = XcisesTable.Id
WHERE RoutineAttribute = ?;", routineId)).ToList();[24]
                }
                catch
                {
                    return null;
```

[22] Cross-Table SQL Queries
[23] Exception Handling
[24] Cross-Table SQL Queries

```
                }[25]
            }
            public async void DeleteRoutineComponentsAsync(int routineId)
            {
                await DbCon.QueryAsync<RoutineComponentsTable>("DELETE FROM RoutineComponentsTable WHERE RoutineAttribute = ?;", routineId);
            }
        // **** RoutineComponentsTable Methods End ****

        // **** Daily/MonthlyMediansTable Methods Start ****
            public async Task<List<SetMediansTable>> GetSetsGraphDataAsync()
            {
                try
                {
                    return await DbCon.QueryAsync<SetMediansTable>("SELECT * FROM DailyMediansTable UNION SELECT * FROM MonthlyMediansTable
ORDER BY DateAttribute ASC;");[26]
                }
                catch
                {
                    return null;
                }[27]
            }
            public async Task<List<SetMediansTable>> GetXciseMonthlyMediansAsync(int xciseId)
            {
                try
                {
                    return await DbCon.QueryAsync<SetMediansTable>("SELECT E1RMaxAttribute, DateAttribute FROM MonthlyMediansTable WHERE
XciseIdAttribute = ? ORDER BY DateAttribute ASC;", xciseId);
                }
```

[25] Exception Handling
[26] Cross-Table SQL Queries
[27] Exception Handling

```
                catch
                {
                    return null;
                }[28]
            }
        // **** Daily/MonthlyMediansTable Methods End ****

        // **** UserDataTable Methods Start ****
            public async Task<List<UserDataTable>> GetBodyweightGraphDataAsync()
            {
                try
                {
                    return await DbCon.QueryAsync<UserDataTable>("SELECT BodyweightAttribute, DateAttribute FROM UserDataTable;");
                }
                catch
                {
                    return null;
                }[29]
            }

            public async Task<List<UserDataTable>> GetConsistencyGraphDataAsync()
            {
                try
                {
                    return await DbCon.QueryAsync<UserDataTable>("SELECT WeeklyConsistencyAttribute, DateAttribute FROM UserDataTable WHERE
DateAttribute < ?;", Utilities.GetLastMonday());
                }
                catch
                {
```

[28] Exception Handling
[29] Exception Handling

```
                    return null;
                }[30]
            }
            public async Task<bool> GetIsBodyweightXciseAsync(int id)
            {
                return (await DbCon.QueryAsync<XcisesTable>("SELECT IsBodyweightAttribute FROM XcisesTable WHERE Id = ?;",
id))[0].IsBodyweightAttribute;
            }

            public async void UpdateBodyweightThisWeekAsync(float bodyweight)
            {
                await DbCon.QueryAsync<UserDataTable>("UPDATE UserDataTable SET BodyweightAttribute = ? WHERE DateAttribute = ?;",
bodyweight, Utilities.GetLastMonday());
            }

            public async void UpdateConsistencyGoalThisWeekAsync(int consistency)
            {
                await DbCon.QueryAsync<UserDataTable>("UPDATE UserDataTable SET WeeklyConsistencyGoalAttribute = ? WHERE DateAttribute = ?;",
consistency, Utilities.GetLastMonday());
            }

            public async Task<UserDataTable> GetThisWeeksUserDataAsync()
            {
                await CheckNeedNewUserDataAsync();
                try
                {
                return (await DbCon.QueryAsync<UserDataTable>("SELECT * FROM UserDataTable ORDER BY DateAttribute DESC;"))[0];

                }
```

[30] Exception Handling

```
            catch
            {
                return null;
            }[31]
        }

        public async Task<List<UserDataTable>> GetUserDataAsync()
        {
            try
            {
                return await DbCon.QueryAsync<UserDataTable>("SELECT * FROM UserDataTable ORDER BY DateAttribute DESC;");
            }
            catch
            {
                return null;
            }[32]
        }


        public async Task CheckNeedNewUserDataAsync()[33]
        {
        List<UserDataTable> userdata;
        try
            {
            userdata = await DbCon.QueryAsync<UserDataTable>("SELECT * FROM UserDataTable ORDER BY DateAttribute DESC;");
            }
        catch
            {
```

[31] Exception Handling
[32] Exception Handling
[33] Self-Defined Algorithms

```
            userdata = null;
            }
            // can't use GetUserDataAsync as it calls this method
        if (userdata == null || userdata.Count() == 0)
        {
            // if no userdata for this week

            await DbCon.InsertAsync(new UserDataTable
            {
                BodyweightAttribute = 0,
                DateAttribute = Utilities.GetLastMonday(),
                WeeklyConsistencyAttribute = 0,
                WeeklyConsistencyGoalAttribute = 0
            });
        } else if (userdata[0].DateAttribute < Utilities.GetLastMonday())
        {
            await DbCon.InsertAsync(new UserDataTable
            {
                BodyweightAttribute = userdata[0].BodyweightAttribute,
                DateAttribute = Utilities.GetLastMonday(),
                WeeklyConsistencyAttribute = 0,
                WeeklyConsistencyGoalAttribute = userdata[0].WeeklyConsistencyGoalAttribute
            });
        }
    }

        public async void UpdateWeeklyConsistencyAsync(List<PendingSetsTable> newSets)[34]
        {
            List<DateTime> weeksList = newSets.Select(obj => Utilities.GetLastMonday()).Distinct().ToList();
```

[34] Self-Defined Algorithms

```
                List<UserDataTable> userdataList = await GetUserDataAsync();
                foreach (DateTime week in weeksList)
                {
                    DateTime LB = week;
                    DateTime UB = LB.AddDays(7);
                    int count = newSets.Where(obj => obj.DateAttribute >= LB && obj.DateAttribute < UB).Count();
                    List<UserDataTable> userdataThatWeek = userdataList.Where(obj => obj.DateAttribute == week).ToList();

                    int prevCount;
                    if (userdataThatWeek.Count() != 0) { prevCount = userdataThatWeek[0].WeeklyConsistencyAttribute; } else { prevCount = 0;
}
                    await DbCon.QueryAsync<UserDataTable>("UPDATE UserDataTable SET WeeklyConsistencyAttribute = ? WHERE DateAttribute = ?;",
count + prevCount, week);
                }
            }

            public async void UpdateXcisePBAsync(int xcise, float bestSet)
            {
                float currPB = (await DbCon.QueryAsync<XcisesTable>("SELECT PBAttribute FROM XcisesTable WHERE Id = ?;",
xcise))[0].PBAttribute;
                if (bestSet > currPB)
                {
                    await DbCon.QueryAsync<XcisesTable>("UPDATE XcisesTable SET PBAttribute = ? WHERE Id = ?;", bestSet, xcise);
                }
            }
        // **** UserDataTable Methods End ****
    }
}
```

Utilities.cs

```
using System;
using System.Collections.Generic;
```

```
public static class Utilities
{
     public static float GetMedian (List<float> data)
    {
        // data.Sort();
        // DATA SHOULD ALREADY BE SORTED (DESC or ASC)
        int len = data.Count;
        if (len % 2 == 0)
        {
            return (data[(len / 2) - 1] + data[len / 2]) / 2;
        }
        else
        {
            return  data[(len - 1) / 2];
        }
    }

    public static float GetE1RMax(float reps, float weight)
    {
        if (1 <= reps && reps < 7.614)
        {
            return weight * 36 / (37 - reps);
        }
        else
        {
            return weight * Convert.ToSingle(Math.Pow(reps, 0.1));
        }[35]
        // 1RM = w * (36/(37-r)) is the Brzyki Formula. It is more accurate for 1 <= r < 7.614
        // 1RM = w * r^0.1 is the Lombardi Formula. It is more accurate for r < 1 U r >= 7.614
```

[35] Mathematical Models

```
        // 7.614 and 1 are the intersections between the graphs, chosen as these ranges match my research.
    }

    public static DateTime GetLastMonday()
    {
        int dayOfWeek = Convert.ToInt16(DateTime.Today.DayOfWeek); // Sun (0) to Sat (6)
        if (dayOfWeek == 0)
        {
            return DateTime.Today.AddDays(-6);
        }
        return DateTime.Today.AddDays(1 - dayOfWeek);
    }

}
```

Data Models Folder

GeneralDataModel.cs

```
namespace Fitness_Appv2.Services
{
    abstract public class GeneralDataModel {[36]
        // serves as a template for display data models that are required because the table contains xciseId (which needs to be translated to
an xciseName)
        public int Id { get; set; }
        public int XciseIdAttribute { get; set; }
        public string XciseNameAttribute { get; set; }
    }
}
```

[36] Self-Defined use of OOP

DisplayComponentsDataModel.cs

```
namespace Fitness_Appv2.Services
{
    public class DisplayComponentsDataModel:GeneralDataModel[37]
    {
        public int SetsAttribute { get; set; }
        public int RoutineAttribute { get; set; }
    }
}
```

DisplaySetsDataModel.cs

```
using System;

namespace Fitness_Appv2.Services
{
    public class DisplaySetsDataModel:GeneralDataModel[38]
    {
        public float E1RMaxAttribute { get; set; }
        public DateTime DateAttribute { get; set; }
        public bool DailyMedianTakenAttribute { get; set; }
    }
}
```

LogRoutineComponentDataModel.cs

```
using System.Collections.Generic;

namespace Fitness_Appv2.Services
{
```

[37] Self-Defined use of OOP
[38] Self-Defined use of OOP

```
    public class LogRoutineComponentDataModel:GeneralDataModel[39]
    {
        public int SetsAttribute { get; set; }
        public List<SetLogDataModel> SetsList { get; set; }[40]
    }
}
```

SetLogDataModel.cs

```
namespace Fitness_Appv2.Services
{
    public class SetLogDataModel {
        public int Index { get; set; }
        public float Reps { get; set; }
        public float Weight{ get; set; }
    }
}
```

Tables Folder

Table.cs

```
using SQLite;

namespace Fitness_Appv2.Services
{
    abstract public class Table
    {
        [PrimaryKey, AutoIncrement]
        public int Id { get; set; }
```

[39] Self-Defined use of OOP
[40] Data Model

```
        // This is a property, {get; set;} is just shorthand for a method that retrieves/allocates the value of the property
    }
}
```

SetsTable.cs

```
using System;

namespace Fitness_Appv2.Services
{
   abstract public class SetsTable:Table[41]
    {
        public int XciseIdAttribute { get; set; }[42] // FOREIGN KEY
        public DateTime DateAttribute { get; set; }
        public float E1RMaxAttribute { get; set; }
    }
}
```

DailyMediansTable.cs

```
namespace Fitness_Appv2.Services
{
    public class DailyMediansTable:SetMediansTable { }[43]
}
```

MonthlyMediansTable.cs

```
namespace Fitness_Appv2.Services
{
    public class MonthlyMediansTable:SetMediansTable { }[44]
}
```

[41] Self-Defined use of OOP
[42] Data Model
[43] Self-Defined use of OOP
[44] Self-Defined use of OOP

PendingSetsTable.cs

```
namespace Fitness_Appv2.Services
{
     public class PendingSetsTable:SetsTable[45]
    {
        public bool DailyMedianTakenAttribute { get; set; }
        // need this since record needs to be kept after daily median taken for monthly median
    }
}
```

RoutineComponentsTable.cs

```
namespace Fitness_Appv2.Services
{
    public class RoutineComponentsTable:Table {[46]
        public int XciseIdAttribute { get; set; } // FOREIGN KEY
        public int SetsAttribute {  get; set; }
        public int RoutineAttribute { get; set; }[47]  // FOREIGN KEY
    }
}
```

RoutinesTable.cs

```
namespace Fitness_Appv2.Services
{
    public class RoutinesTable:Table[48]
    {
        public string NameAttribute { get; set; }
    }
}
```

[45] Self-Defined use of OOP
[46] Self-Defined use of OOP
[47] Data Model
[48] Self-Defined use of OOP

SetMediansTable.cs

```
namespace Fitness_Appv2.Services
{
    public class SetMediansTable:SetsTable[49] // need to inherit as need to be able to create objects (when combining daily & monthly for graph)
    {
    }
}
```

UserDataTable.cs

```
 using System;

namespace Fitness_Appv2.Services
{
    public class UserDataTable:Table {[50]
        public DateTime DateAttribute { get; set; }
        public float BodyweightAttribute { get; set; }
        public int WeeklyConsistencyAttribute { get; set; }
        public int WeeklyConsistencyGoalAttribute { get; set; }
    }
}
```

XcisesTable.cs

```
namespace Fitness_Appv2.Services
{
    public class XcisesTable:Table {[51]
        public string XciseNameAttribute { get; set; }
        public bool IsBodyweightAttribute { get; set; }
        public float PBAttribute { get; set; }
```

[49] Self-Defined use of OOP
[50] Self-Defined use of OOP
[51] Self-Defined use of OOP

```
        public float GoalAttribute { get; set; }
        public bool IsPinnedAttribute { get; set; }
    }
}
```

Views Folder

Home.xaml

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="Fitness_Appv2.Views.Home"
             xmlns:vm="clr-namespace:Fitness_Appv2.ViewModels"
             Title="{Binding Title}">
    <!-- **** TEMPLATE CODE START **** -->
    <ContentPage.BindingContext>
        <vm:HomeViewModel />
    </ContentPage.BindingContext>
    <!-- **** TEMPLATE CODE END **** -->

    <Grid RowDefinitions="Auto, Auto, *">
        <Label Text="Welcome" TextColor="Black" FontSize="30" Grid.Row="0"/>

        <Label Text="Pinned Exercises:" TextColor="Black" FontSize="20" Grid.Row="1"/>
        <CollectionView x:Name="viewPinned" Grid.Row="2">
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <StackLayout>
                        <Label Text="ID"/>
                        <Label Text="{Binding Id}"
                               TextColor="Black"
                               FontSize="Medium"/>
                        <Label Text="Name"/>
```

```
                        <Label Text="{Binding XciseNameAttribute}"
                               TextColor="Black"
                               FontSize="Medium"/>
                        <Label Text="Goal"/>
                        <Label Text="{Binding GoalAttribute}"
                               TextColor="Black"
                               FontSize="Medium"/>
                        <Label Text="PB"/>
                        <Label Text="{Binding PBAttribute}"
                               TextColor="Black"
                               FontSize="Medium"/>
                    </StackLayout>
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>
    </Grid>
</ContentPage>
```

Home.xaml.cs

```
using Xamarin.Forms;

namespace Fitness_Appv2.Views
{
    public partial class Home : ContentPage
    {
        public Home()
        {
            InitializeComponent();
        }

        protected override async void OnAppearing()
        {
```

```
            base.OnAppearing();
            viewPinned.ItemsSource = await App.Db.GetPinnedXcisesAsync();
        }
    }
}
```

AddSets.xaml

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="Fitness_Appv2.Views.AddSets">
    <StackLayout>
        <Grid RowDefinitions="Auto,Auto,Auto,Auto,Auto,*,40,Auto">
            <Picker x:Name="xcisePicker"
                    Title="Select Exercise"
                    ItemDisplayBinding="{Binding XciseNameAttribute}"
                    Grid.Row="0"/>
            <Entry x:Name="repsInput" Placeholder="Input Reps" Keyboard="Numeric" Grid.Row="1"/>
            <Entry x:Name="weightInput" Placeholder="Input Weight" Keyboard="Numeric" Grid.Row="2"/>
            <Button x:Name="submitSet" Text="Submit Set" Clicked="SubmitSet_ClickedAsync" Grid.Row="3"/>
            <Label x:Name="inputObjection" IsVisible="False" Grid.Row="4"/>
            <CollectionView x:Name="DataView" BackgroundColor="White" Grid.Row="5" >
                <CollectionView.ItemTemplate>
                    <DataTemplate>
                        <StackLayout>
                            <Label Text="ID"/>
                                <Label Text="{Binding Id}"
                                   TextColor="Black"
                                   FontSize="Medium"/>
                            <Label Text="Exercise"/>
                            <Label Text="{Binding XciseNameAttribute}"
                                   TextColor="Black"
```

```
                                FontSize="Medium"/>
                            <Label Text="Date"/>
                                <Label Text="{Binding DateAttribute}"
                                   TextColor="Black"
                                   FontSize="Medium"/>
                            <Label Text="e1RM"/>
                                <Label Text="{Binding E1RMaxAttribute}"
                                    TextColor="Black"
                                    FontSize="Medium"/>
                            <Label Text="DailyMedianTaken"/>
                            <Label Text="{Binding DailyMedianTakenAttribute}"
                                    TextColor="Black"
                                    FontSize="Medium"/>
                            <Label Text="-----------------------------------" FontSize="Medium" TextColor="Black"/>
                        </StackLayout>
                    </DataTemplate>
                </CollectionView.ItemTemplate>
            </CollectionView>
            <Grid ColumnDefinitions="Auto, Auto" Grid.Row="6">
                <Button x:Name="UndoSet" Text="Undo" Clicked="UndoSet_ClickedAsync" Grid.Column="0"/>
                <Button x:Name="RedoSet" Text="Redo" Clicked="RedoSet_ClickedAsync" Grid.Column="1"/>
            </Grid>
        </Grid>
    </StackLayout>
</ContentPage>
```

AddSets.xaml.cs

```
using Fitness_Appv2.Services;
using System;
using System.Threading.Tasks;
using Xamarin.Forms;
```

```
namespace Fitness_Appv2.Views
{
    public partial class AddSets : ContentPage
    {
        public AddSets()
        {
            InitializeComponent();
        }
        protected override async void OnAppearing()
        {
            base.OnAppearing();
            DataView.ItemsSource = await App.Db.GetPendingSetsToViewAsync();
            xcisePicker.ItemsSource = await App.Db.GetXciseNamesAsync();
        }
        private async void SubmitSet_ClickedAsync(object sender, EventArgs e)
        {
            if (submitSet.Text == "Submit Set")[52] // this is to prevent double clicks registering twice
            {
                submitSet.Text = "Submitting ...";
                inputObjection.IsVisible = false;
                XcisesTable item = xcisePicker.SelectedItem as XcisesTable;
                int xciseId = item.Id; // record Index instead?
                bool xciseIsBodyweight = await App.Db.GetIsBodyweightXciseAsync(xciseId);
                float reps = Convert.ToSingle(repsInput.Text);
                float weight;
                if (!xciseIsBodyweight) {
                    weight = Convert.ToSingle(weightInput.Text);
                }
                else
```

[52] Defensive Programming

```
{
    UserDataTable userdata = (await App.Db.GetThisWeeksUserDataAsync());
    if (userdata != null && userdata.BodyweightAttribute != 0)[53]
    {
        weight = userdata.BodyweightAttribute + Convert.ToSingle(weightInput.Text);
    } // If it is a bodyweight exercise, must add bodyweight to any additional weight
    else
    {
        inputObjection.IsVisible = true;
        inputObjection.Text = "This is a Bodyweight Exercise. Please record your bodyweight on the home page.";
        await Task.Delay(1500);
        submitSet.Text = "Submit Set";
        return;
    }
}

if ((reps > 0) && (weight > 0))[54]
{
    DateTime date = DateTime.Today;
    float e1RMax = Utilities.GetE1RMax(reps, weight);
    await App.Db.SaveSetAsync(new PendingSetsTable
    {
        XciseIdAttribute = xciseId,
        DateAttribute = date,
        E1RMaxAttribute = e1RMax,
        DailyMedianTakenAttribute = false,
    }); // passes record obj into save method
    DataView.ItemsSource = null;
    DataView.ItemsSource = await App.Db.GetPendingSetsToViewAsync();
```

[53] Defensive Programming
[54] Defensive Programming

```
                    repsInput.Text = weightInput.Text = "";
                    xcisePicker.SelectedIndex = -1;
                }
                else
                {
                    inputObjection.IsVisible = true;
                    inputObjection.Text = "Reps and Weight must be above 0 if it is not a Bodyweight Exercise";
                    return;
                }
                await Task.Delay(1500);
                submitSet.Text = "Submit Set";
            }
        }
        private async void UndoSet_ClickedAsync(object sender, EventArgs e)
        {
            App.Db.UndoSetAsync();
            DataView.ItemsSource = null;
            DataView.ItemsSource = await App.Db.GetPendingSetsToViewAsync();
        }
        private async void RedoSet_ClickedAsync(object sender, EventArgs e)
        {
            App.Db.RedoSetAsync();
            DataView.ItemsSource = null;
            DataView.ItemsSource = await App.Db.GetPendingSetsToViewAsync();
        }
    }
}
```

Stats.xaml

```
<?xml version="1.0" encoding="utf-8" ?>

<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"
```

```
          xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
          xmlns:chart="clr-namespace:Syncfusion.SfChart.XForms;assembly=Syncfusion.SfChart.XForms"
          x:Class="Fitness_Appv2.Views.Stats">
    <Grid RowDefinitions="Auto, Auto,*">
        <Picker x:Name="primaryPicker"
            Title="Select Graph Type"
            SelectedIndexChanged="PrimaryPicker_Changed"
            Grid.Row="0" />
        <Picker x:Name="secondaryPicker"
            Title="Select Variable"
            SelectedIndexChanged="SecondaryPicker_Changed"
            IsEnabled="False"
            Grid.Row="1"/>

        <chart:SfChart x:Name="Chart" Grid.Row="2">

            <chart:SfChart.PrimaryAxis>
                <chart:DateTimeAxis/>
            </chart:SfChart.PrimaryAxis>

            <chart:SfChart.SecondaryAxis>
                <chart:NumericalAxis>
                </chart:NumericalAxis>
            </chart:SfChart.SecondaryAxis>

            <chart:SfChart.Series>
                <chart:LineSeries
                    x:Name="chartSeries"
                    XBindingPath="DateAttribute"/>
            </chart:SfChart.Series>
```

```
            <chart:SfChart.ChartBehaviors>
                <chart:ChartZoomPanBehavior EnablePanning ="true" EnableDoubleTap="true"/> <!--DoubleTap allows to zoom in -->
            </chart:SfChart.ChartBehaviors>
        <!--Could also add zoom factor-->
        </chart:SfChart>
    </Grid>
</ContentPage>
```

Stats.xaml.cs

```
using Fitness_Appv2.Services;
using System;
using System.Collections.Generic;
using System.Linq;
using Xamarin.Forms;

namespace Fitness_Appv2.Views
{
    public partial class Stats : ContentPage
    {
        List<SetMediansTable> setsGraphSource;
        List<UserDataTable> bodyweightGraphSource;
        List<UserDataTable> consistencyGraphSource;

        List<string> userDataSecondaryOptions;
        List<XcisesTable> xciseSecondaryOptions;[55]

        string primaryChoice;
        XcisesTable xciseSecondaryChoice;
        string userDataSecondaryChoice;
```

[55] Data Structures

```
public Stats()
{
    InitializeComponent();
}
protected override async void OnAppearing()
{
    base.OnAppearing();
    primaryPicker.ItemsSource = new List<string>() { "Exercises", "User Data" }; // provides data setsGraphSource for picker

    setsGraphSource = await App.Db.GetSetsGraphDataAsync();
    bodyweightGraphSource = await App.Db.GetBodyweightGraphDataAsync();
    consistencyGraphSource = await App.Db.GetConsistencyGraphDataAsync();

    userDataSecondaryOptions = new List<string>() { "Bodyweight", "Consistency" };
    xciseSecondaryOptions = await App.Db.GetXcisesAsync();
}
private void PrimaryPicker_Changed (object sender, EventArgs e)
{
    primaryChoice = primaryPicker.SelectedItem as string;

    if (primaryChoice == "Exercises")
    {
        secondaryPicker.ItemsSource = xciseSecondaryOptions;
        secondaryPicker.ItemDisplayBinding = new Binding("XciseNameAttribute");[56]
    }
    else if (primaryChoice == "User Data")
    {
        secondaryPicker.ItemsSource = userDataSecondaryOptions;
        secondaryPicker.ItemDisplayBinding = null;[57]
```

[56] Data Structures
[57] Data Structures

```
        }
        secondaryPicker.IsEnabled = true;
    }
    private void SecondaryPicker_Changed (object sender, EventArgs e)
    {
        primaryChoice = primaryPicker.SelectedItem as string;
        if (primaryChoice == "Exercises")
        {
            xciseSecondaryChoice = secondaryPicker.SelectedItem as XcisesTable;
            if (xciseSecondaryChoice != null)[58]
            {
                chartSeries.ItemsSource = setsGraphSource.Where(obj => obj.XciseIdAttribute == xciseSecondaryChoice.Id);
                chartSeries.YBindingPath = "E1RMaxAttribute";[59]
            }
        }
        else if (primaryChoice == "User Data")
        {
            userDataSecondaryChoice = secondaryPicker.SelectedItem as string;
            [60]if (userDataSecondaryChoice != null)
            {
                if (userDataSecondaryChoice == "Bodyweight")
                {
                    chartSeries.ItemsSource = bodyweightGraphSource;
                    chartSeries.YBindingPath = "BodyweightAttribute";[61]
                }
                else if (userDataSecondaryChoice == "Consistency")
                {
```

[58] Defensive Programming
[59] Data Structures
[60] Defensive Programming
[61] Data Structures

```
                    chartSeries.ItemsSource = consistencyGraphSource;
                    chartSeries.YBindingPath = "WeeklyConsistencyAttribute";[62]
                }
            }
        }
    }
}
```

Routines.xaml

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="Fitness_Appv2.Views.Routines">
    <StackLayout>
        <Grid RowDefinitions="*, Auto, Auto">
            <CollectionView x:Name="RoutinesView" SelectionMode="Single" SelectionChanged="RoutinesView_SelectionChanged"
BackgroundColor="White" Grid.Row="0">
                <CollectionView.ItemTemplate>
                    <DataTemplate>
                        <StackLayout>
                            <Label Text="{Binding Id}"/>
                            <Label Text="{Binding NameAttribute}"/>
                        </StackLayout>
                    </DataTemplate>
                </CollectionView.ItemTemplate>
            </CollectionView>

            <Grid x:Name = "RoutineOptions" RowDefinitions= "Auto, Auto, Auto" IsVisible="False" Grid.Row="1">
                <Button x:Name="editRoutine" Text="Edit" Clicked="EditRoutine_Clicked" Grid.Row="0"/>
```

[62] Data Structures

```
            <Button x:Name="deleteRoutine" Text="Delete" Clicked="DeleteRoutine_Clicked" Grid.Row="1"/>
            <Button x:Name="logRoutine" Text="Log" Clicked="LogRoutine_Clicked" Grid.Row="2"/>
        </Grid>

        <Button x:Name="newRoutine" Text = "Define New Routine" Clicked="NewRoutine_Clicked" Grid.Row="2"/>
    </Grid>
</StackLayout>
</ContentPage>
```

Routines.xaml.cs

```
using Fitness_Appv2.Services;
using System;
using System.Collections.Generic;
using Xamarin.Forms;

namespace Fitness_Appv2.Views
{
    public partial class Routines : ContentPage
    {
        List<RoutinesTable> displayedRoutines;
        int currRoutineIndex;[63]
        public Routines()
        {
            InitializeComponent();
        }
        protected override async void OnAppearing()
        {
            base.OnAppearing();
            displayedRoutines = await App.Db.GetRoutinesAsync();
            RoutinesView.ItemsSource = displayedRoutines;
```

[63] Data Structures

```
        }

        private void RoutinesView_SelectionChanged(object sender, SelectionChangedEventArgs e)
        {
            currRoutineIndex = displayedRoutines.IndexOf(e.CurrentSelection[0] as RoutinesTable);[64]
            RoutineOptions.IsVisible = true;
        }
        private void EditRoutine_Clicked(object sender, EventArgs e)
        {
            Navigation.PushAsync(new EditRoutine(displayedRoutines[currRoutineIndex].Id));
        }

        private void DeleteRoutine_Clicked(object sender, EventArgs e)
        {
            App.Db.DeleteRoutineComponentsAsync(displayedRoutines[currRoutineIndex].Id);
            App.Db.DeleteRoutineAsync(displayedRoutines[currRoutineIndex].Id);
            displayedRoutines.RemoveAt(currRoutineIndex);[65]
            RoutineOptions.IsVisible = false;
            RoutinesView.ItemsSource = null;
            RoutinesView.ItemsSource = displayedRoutines;
        }

        private void LogRoutine_Clicked(object sender, EventArgs e)
        {
            Navigation.PushAsync(new LogRoutine(displayedRoutines[currRoutineIndex].Id));
        }

        private void NewRoutine_Clicked(object sender, EventArgs e)
```

[64] Data Structures

[65] Data Structures

```
        {
            Navigation.PushAsync(new EditRoutine(0));
        }


    }
}
```

EditRoutine.xaml

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="Fitness_Appv2.Views.EditRoutine">
    <StackLayout>
        <Grid RowDefinitions="Auto, *, Auto, Auto, Auto, Auto">
            <Entry x:Name = "RoutineName" Grid.Row="0"/>
            <CollectionView x:Name="ComponentsView" SelectionMode="Single" SelectionChanged="ComponentsView_SelectionChanged"
BackgroundColor="White" Grid.Row="1">
                <CollectionView.ItemTemplate>
                    <DataTemplate>
                        <StackLayout>
                            <Label Text="{Binding XciseNameAttribute}"/>
                            <Label Text="{Binding SetsAttribute}"/>
                        </StackLayout>
                    </DataTemplate>
                </CollectionView.ItemTemplate>
            </CollectionView>
            <Grid x:Name = "ComponentEdits" RowDefinitions= "Auto, Auto, Auto, Auto, Auto" IsVisible="False" Grid.Row="2">
                <Picker x:Name="xcisePicker"
                        Title="Select Exercise"
                        ItemDisplayBinding="{Binding XciseNameAttribute}"
                        Grid.Row="0"/>
```

```
                <Entry x:Name="setsInput" Placeholder="Input Sets" Keyboard="Numeric" Grid.Row="1"/>
                <Button x:Name="saveComponent" Text="Save" Clicked="SaveComponent_Clicked" Grid.Row="2"/>
                <Button x:Name="deleteComponent" Text="Delete" Clicked="DeleteComponent_Clicked" Grid.Row="3"/>
            </Grid>
            <Button x:Name="newComponent" Text = "Add Exercise" Clicked="NewComponent_Clicked" Grid.Row="3"/>
            <Button x:Name= "saveRoutine" Text="Save Routine" Clicked="SaveRoutine_ClickedAsync" Grid.Row="4"/>
            <Label x:Name="inputObjection" IsVisible="False" Grid.Row="5"/>
        </Grid>
    </StackLayout>
</ContentPage>
```

EditRoutine.xaml.cs

```
using Fitness_Appv2.Services;
using System;
using System.Collections.Generic;
using System.Linq;
using Xamarin.Forms;

namespace Fitness_Appv2.Views
{
    public partial class EditRoutine : ContentPage
    {
        List<DisplayComponentsDataModel> activeComponents;
        int currComponentIndex;[66]
        int RoutineId;
        public EditRoutine(int inputRoutineId)
        {
            InitializeComponent();
            RoutineId = inputRoutineId;
        }
```

[66] Data Structures

```
protected override async void OnAppearing()
{
    base.OnAppearing();
    ComponentEdits.IsVisible = false;
    xcisePicker.ItemsSource = await App.Db.GetXciseNamesAsync();
    if (RoutineId == 0)
    {
        activeComponents = new List<DisplayComponentsDataModel>() { new DisplayComponentsDataModel() { Id = 1 } };
    }
    else
    {
        RoutineName.Text = await App.Db.GetRoutineNameAsync(RoutineId);
        activeComponents = await App.Db.GetRoutineComponentsToEditAsync(RoutineId);
    }
    ComponentsView.ItemsSource = activeComponents;
}
private void NewComponent_Clicked(object sender, EventArgs e)
{
    if (activeComponents.Count() == 0)
    {
        activeComponents.Add(new DisplayComponentsDataModel() { Id = 1 });[67] [68]
    }
    else
    {
        activeComponents.Add(new DisplayComponentsDataModel() { Id = activeComponents.Last().Id + 1 });[69] [70]
    }
    ComponentsView.ItemsSource = null; // Resets ItemsSource so Display notices change
```

[67] Data Structures
[68] Dynamic Generation of Objects
[69] Data Structures
[70] Dynamic Generation of Objects

```
            ComponentsView.ItemsSource = activeComponents;
        }
        private void ComponentsView_SelectionChanged(object sender, SelectionChangedEventArgs e)
        {
            currComponentIndex = activeComponents.IndexOf(e.CurrentSelection[0] as DisplayComponentsDataModel);[71]
            ComponentEdits.IsVisible = true;
            xcisePicker.SelectedIndex = activeComponents[currComponentIndex].XciseIdAttribute - 1; // SelectedIndex starts from 0,
XciseIdAttribute starts from 1 (SelectedIndex = -1 --> xcisePicker not set)
            if (activeComponents[currComponentIndex].SetsAttribute != 0)
            {
                setsInput.Text = Convert.ToString(activeComponents[currComponentIndex].SetsAttribute);
            }
            else
            {
                setsInput.Text = "";
            }
        }
        private void SaveComponent_Clicked(object sender, EventArgs e)
        {
            XcisesTable item = xcisePicker.SelectedItem as XcisesTable;
            int xciseId = item.Id;
            string xciseName = item.XciseNameAttribute;
            int sets = Convert.ToInt16(setsInput.Text);
            activeComponents[currComponentIndex].XciseIdAttribute = xciseId;
            activeComponents[currComponentIndex].XciseNameAttribute = xciseName;
            activeComponents[currComponentIndex].SetsAttribute = sets;[72]
            ComponentsView.ItemsSource = null;
            ComponentsView.ItemsSource = activeComponents;
            ComponentEdits.IsVisible = false;
```

[71] Data Structures
[72] Data Structures

```
}
private void DeleteComponent_Clicked(object sender, EventArgs e)
{
    activeComponents.RemoveAt(currComponentIndex);[73]
    ComponentEdits.IsVisible = false;
    ComponentsView.ItemsSource = null;
    ComponentsView.ItemsSource = activeComponents;
}
private async void SaveRoutine_ClickedAsync(object sender, EventArgs e)
{
    if (string.IsNullOrEmpty(RoutineName.Text))[74]
    {
        inputObjection.IsVisible = true;
        inputObjection.Text = "Routine Must Have A Name";
        return;
    }
    foreach (DisplayComponentsDataModel component in activeComponents)
    {
        if (component.SetsAttribute <= 0 || component.XciseIdAttribute == 0)[75]
        {
            inputObjection.IsVisible = true;
            inputObjection.Text = "Num. SetsAttribute must be above 0, You must choose an exercise for each component.";
            return;
        }
    }
    inputObjection.IsVisible = false;

    if (RoutineId == 0) // i.e. if Routine is new
```

[73] Data Structures
[74] Defensive Programming
[75] Defensive Programming

```
            {
                RoutinesTable newRoutine = new RoutinesTable { NameAttribute = RoutineName.Text };

                RoutineId = await App.Db.SaveRoutineAsync(newRoutine);
            }
            else
            {
                App.Db.UpdateRoutineNameAsync(RoutineName.Text, RoutineId);
                App.Db.DeleteRoutineComponentsAsync(RoutineId); // clears previous version of routine
            }
            foreach (DisplayComponentsDataModel component in activeComponents)
            {
                component.Id = 0; // reset to 0 so autoIncrement is triggered (autoIncrement starts from 1)
                component.RoutineAttribute = RoutineId;
                App.Db.SaveRoutineComponentAsync(new RoutineComponentsTable() { RoutineAttribute = component.RoutineAttribute, SetsAttribute
= component.SetsAttribute, XciseIdAttribute = component.XciseIdAttribute});
            }
            await Navigation.PopAsync();

        }
    }
}
```

LogRoutine.xaml

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="Fitness_Appv2.Views.LogRoutine">
    <StackLayout>
        <Grid RowDefinitions="*, Auto, Auto">
            <Grid x:Name="AlterRoutine" RowDefinitions="*, *, Auto" Grid.Row="0">
```

```
                <CollectionView x:Name="ComponentsView" SelectionMode="Single" SelectionChanged="ComponentsView_SelectionChanged"
BackgroundColor="White" Grid.Row="0">
                    <CollectionView.ItemTemplate>
                        <DataTemplate>
                            <StackLayout>
                                <Label Text="{Binding XciseNameAttribute}"/>
                                <Label Text="{Binding SetsAttribute}"/>
                            </StackLayout>
                        </DataTemplate>
                    </CollectionView.ItemTemplate>
                </CollectionView>

                <Grid x:Name = "ComponentEdits" RowDefinitions= "Auto, Auto, Auto, Auto" IsVisible="False" Grid.Row="1">
                    <Picker x:Name="xcisePicker"
                            Title="Select Exercise"
                            ItemDisplayBinding="{Binding XciseNameAttribute}"
                            Grid.Row="0"/>
                    <Entry x:Name="setsInput" Placeholder="Input Sets" Keyboard="Numeric" Grid.Row="1"/>
                    <Button x:Name="deleteComponent" Text="Delete" Clicked="DeleteComponent_Clicked" Grid.Row="2"/>
                    <Button x:Name="defineComponent" Text="Define Sets" Clicked="DefineComponent_Clicked" Grid.Row="3"/>
                </Grid>
                <Button x:Name="newComponent" Text = "Add Exercise" Clicked="NewComponent_Clicked" Grid.Row="2"/>
            </Grid>

            <Grid x:Name="SetsData" RowDefinitions ="*, Auto, Auto" IsVisible="False" Grid.Row="0">
                <CollectionView x:Name="SetsView" SelectionMode="Single" SelectionChanged="SetsView_SelectionChanged" BackgroundColor="White"
Grid.Row="0">
                    <CollectionView.ItemTemplate>
                        <DataTemplate>
                            <StackLayout>
                                <Label Text="{Binding Reps}"/>
```

```
                            <Label Text="{Binding Weight}"/>
                        </StackLayout>
                    </DataTemplate>
                </CollectionView.ItemTemplate>
            </CollectionView>
            <Grid x:Name = "SetEdits" RowDefinitions= "Auto, Auto, Auto" IsVisible="False" Grid.Row="1">
                <Entry x:Name="repsInput" Placeholder = "Input Reps" Keyboard="Numeric" Grid.Row="0"/>
                <Entry x:Name="weightInput" Placeholder= "Input Weight" Keyboard="Numeric" Grid.Row="1"/>
                <Button x:Name="saveSet" Text = "Save" Clicked = "SaveSet_Clicked" Grid.Row="2"/>
            </Grid>
            <Button x:Name="editFinished" Text="Back" Clicked="EditFinished_Clicked" Grid.Row="2"/>
        </Grid>
        <Button x:Name= "logWorkout" Text="Log Workout" Clicked="LogWorkout_ClickedAsync" Grid.Row="1"/>
        <Label x:Name="inputObjection" IsVisible="False" Grid.Row="2"/>
    </Grid>
</StackLayout>
</ContentPage>
```

LogRoutine.xaml.cs

```
using Fitness_Appv2.Services;
using System;
using System.Collections.Generic;
using System.Linq;
using Xamarin.Forms;

namespace Fitness_Appv2.Views
{
    public partial class LogRoutine : ContentPage
    {
        List<LogRoutineComponentDataModel> activeComponents;
        int currComponentIndex;
```

```
        int currSetIndex;[76]
        readonly int RoutineId;[77]
        public LogRoutine(int inputRoutineId)
        {
            InitializeComponent();
            RoutineId = inputRoutineId;
        }
        protected override async void OnAppearing()
        {
            base.OnAppearing();
            xcisePicker.ItemsSource = await App.Db.GetXciseNamesAsync();
            activeComponents = await App.Db.GetRoutineComponentsToLogAsync(RoutineId);
            ComponentsView.ItemsSource = activeComponents;
        }
        private void NewComponent_Clicked(object sender, EventArgs e)
        {
            if (activeComponents.Count() == 0)
            {
                activeComponents.Add(new LogRoutineComponentDataModel() { Id = 1 });[78] [79]
            } else
            {
                activeComponents.Add(new LogRoutineComponentDataModel() { Id = activeComponents.Last().Id + 1 });[80] [81]
            }
            ComponentsView.ItemsSource = null;
            ComponentsView.ItemsSource = activeComponents;
```

[76] Data Structures
[77] Use of Read-Only Objects
[78] Data Structures
[79] Dynamic Generation of Objects
[80] Data Structures
[81] Dynamic Generation of Objects

```
}
private void ComponentsView_SelectionChanged(object sender, SelectionChangedEventArgs e)
{
    currComponentIndex = activeComponents.IndexOf(e.CurrentSelection[0] as LogRoutineComponentDataModel);[82]
    ComponentEdits.IsVisible = true;
    xcisePicker.SelectedIndex = activeComponents[currComponentIndex].XciseIdAttribute - 1;
    if (activeComponents[currComponentIndex].SetsAttribute != 0)
    {
        setsInput.Text = Convert.ToString(activeComponents[currComponentIndex].SetsAttribute);
    }
    else
    {
        setsInput.Text = "";
    }
}

private void DeleteComponent_Clicked(object sender, EventArgs e)
{
    activeComponents.RemoveAt(currComponentIndex);[83] // tempId not changing when delete therefore transition to index based
    ComponentEdits.IsVisible = false;
    ComponentsView.ItemsSource = null;
    ComponentsView.ItemsSource = activeComponents;
}
private void DefineComponent_Clicked(object sender, EventArgs e)[84]
{
    if (Convert.ToInt16(setsInput.Text) > 0)[85]
    {
```

[82] Data Structures
[83] Data Structures
[84] Self-Defined Algorithm
[85] Defensive Programming

```
                bool setsChanged = false;
                AlterRoutine.IsVisible = false;
                SetsData.IsVisible = true;
                if (activeComponents[currComponentIndex].SetsAttribute != Convert.ToInt16(setsInput.Text))
                {
                    setsChanged = true;
                    activeComponents[currComponentIndex].SetsAttribute = Convert.ToInt16(setsInput.Text);
                }
                activeComponents[currComponentIndex].XciseIdAttribute = xcisePicker.SelectedIndex + 1;
                activeComponents[currComponentIndex].XciseNameAttribute = (xcisePicker.SelectedItem as XcisesTable).XciseNameAttribute;[86]
                if (activeComponents[currComponentIndex].SetsList == null || setsChanged == true)
                {
                    List<SetLogDataModel> expectedInputs = new List<SetLogDataModel>();
                    for (int i = 0; i < activeComponents[currComponentIndex].SetsAttribute; i++)
                    {
                        expectedInputs.Add(new SetLogDataModel { Index = i });
                    }
                    activeComponents[currComponentIndex].SetsList = expectedInputs;[87]
                }
                SetsView.ItemsSource = activeComponents[currComponentIndex].SetsList;
            }
        }

        private void SetsView_SelectionChanged (object sender, SelectionChangedEventArgs e)
        {
            currSetIndex = (e.CurrentSelection[0] as SetLogDataModel).Index;[88]
            SetEdits.IsVisible = true;
            if (activeComponents[currComponentIndex].SetsList[currSetIndex].Reps != 0)
```

[86] Data Structures
[87] Data Structures
[88] Data Structures

```
            {
                repsInput.Text = Convert.ToString(activeComponents[currComponentIndex].SetsList[currSetIndex].Reps);
            }
            else
            {
                repsInput.Text = "";
            }
            if (activeComponents[currComponentIndex].SetsList[currSetIndex].Weight != 0)
            {
                weightInput.Text = Convert.ToString(activeComponents[currComponentIndex].SetsList[currSetIndex].Weight);
            }
            else
            {
                weightInput.Text = "";
            }
        }

        private void SaveSet_Clicked(object sender, EventArgs e)
        {
            float reps;
            if (string.IsNullOrEmpty(repsInput.Text)) { reps = 0; } else { reps = Convert.ToSingle(repsInput.Text); };[89]
            float weight;
            if (string.IsNullOrEmpty(weightInput.Text)) { weight = 0; } else { weight = Convert.ToSingle(weightInput.Text); };[90]
            activeComponents[currComponentIndex].SetsList[currSetIndex].Reps = reps;
            activeComponents[currComponentIndex].SetsList[currSetIndex].Weight = weight;[91]
            SetsView.ItemsSource = null;
            SetsView.ItemsSource = activeComponents[currComponentIndex].SetsList;
            SetEdits.IsVisible = false;
```

[89] Defensive Programming
[90] Defensive Programming
[91] Data Structures

```
}
private void EditFinished_Clicked(object sender, EventArgs e)
{
    SetsData.IsVisible = false;
    AlterRoutine.IsVisible = true;
    ComponentsView.ItemsSource = null;
    ComponentsView.ItemsSource = activeComponents;
}

private async void LogWorkout_ClickedAsync(object sender, EventArgs e)
{
    DateTime date = DateTime.Today;
    UserDataTable userdata = await App.Db.GetThisWeeksUserDataAsync();
    // sanitisation
    foreach (LogRoutineComponentDataModel component in activeComponents)
    {
        bool isBodyweight = await App.Db.GetIsBodyweightXciseAsync(component.XciseIdAttribute);

        if (isBodyweight && userdata.BodyweightAttribute == 0)[92]
        {
            inputObjection.IsVisible = true;
            inputObjection.Text = "Bodyweight Exercise Inputed. Please record your bodyweight on the home page first.";
            return;
        }
        if (component.SetsAttribute <= 0 || component.XciseIdAttribute == 0)[93]
        {
            inputObjection.IsVisible = true;
            inputObjection.Text = "Workout Data Incomplete";
            return;
```

[92] Defensive Programming
[93] Defensive Programming

```
        }

        foreach (SetLogDataModel set in component.SetsList)
        {
            if (isBodyweight) { set.Weight = set.Weight + userdata.BodyweightAttribute; }
            if ((set.Reps <= 0) || (set.Weight <= 0))[94]
            {
                inputObjection.IsVisible = true;
                inputObjection.Text = "Workout Data Incomplete";
                return;
            }
        }
    }
    inputObjection.IsVisible = false;

    // log sets
    foreach (LogRoutineComponentDataModel component in activeComponents)
    {
        bool isBodyweight = await App.Db.GetIsBodyweightXciseAsync(component.XciseIdAttribute);
        foreach (SetLogDataModel set in component.SetsList)
        {
            float e1RMax = Utilities.GetE1RMax(set.Reps, set.Weight);
            await App.Db.SaveSetAsync(new PendingSetsTable
            {
                XciseIdAttribute = component.XciseIdAttribute,
                DateAttribute = date,
                E1RMaxAttribute = e1RMax,
                DailyMedianTakenAttribute = false,
            });
```

[94] Defensive Programming

```
                }
            }
            await Navigation.PopAsync();
        }


    }
}
```

Xcises.xaml

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="Fitness_Appv2.Views.Xcises">
    <StackLayout>
        <Grid RowDefinitions="Auto, Auto, Auto, *, Auto">
            <Entry x:Name="xciseNameInput" Placeholder ="Input Exercise Name" Grid.Row="0"/>
            <Grid ColumnDefinitions="Auto, Auto" Grid.Row="1">
                <Label Text="Bodyweight Exercise?" FontSize ="Medium" TextColor="Black" Grid.Column="0"/>
                <CheckBox x:Name="isBodyweightInput" Grid.Column="1"/>
            </Grid>
            <Button x:Name="submitXcise" Text="Submit Exercise" Clicked="SubmitXcise_ClickedAsync" Grid.Row="2"/>
            <CollectionView x:Name="xcisesView" Grid.Row="3" BackgroundColor="White" SelectionMode="Single"
SelectionChanged="XcisesView_SelectionChanged">
                <CollectionView.ItemTemplate>
                    <DataTemplate>
                        <StackLayout>
                            <Label Text="ID"/>
                            <Label Text="{Binding Id}"
                                TextColor="Black"
                                FontSize="Medium"/>
                            <Label Text="Name"/>
```

```
                        <Label Text="{Binding XciseNameAttribute}"
                            TextColor="Black"
                            FontSize="Medium"/>
                        <Label Text="Is Bodyweight Exercise?"/>
                        <Label Text="{Binding IsBodyweightAttribute}"
                            TextColor="Black"
                            FontSize="Medium"/>
                        <Label Text="Goal"/>
                        <Label Text="{Binding GoalAttribute}"
                            TextColor="Black"
                            FontSize="Medium"/>
                        <Label Text="PB"/>
                        <Label Text="{Binding PBAttribute}"
                            TextColor="Black"
                            FontSize="Medium"/>
                        <Label Text="Pinned?"/>
                        <Label Text="{Binding IsPinnedAttribute}"
                            TextColor="Black"
                            FontSize="Medium"/>
                        <Label Text="-----------------------------------" FontSize="Medium" TextColor="Black"/>
                    </StackLayout>
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>
        <Grid x:Name="changeIsPinned" ColumnDefinitions="Auto, Auto" RowDefinitions="Auto, Auto" IsVisible="False" Grid.Row="4">
            <Label Text="Pin? " Grid.Column="0" Grid.Row="0"/>
            <CheckBox x:Name ="isPinnedInput" Grid.Column="1" Grid.Row="0"/>
            <Button x:Name="submitIsPinned" Text="Save" Clicked="SubmitIsPinned_Clicked" Grid.Row="1"/>
        </Grid>
    </Grid>
</StackLayout>
```

```
</ContentPage>
```

Xcises.xaml.cs

```
using Fitness_Appv2.Services;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Xamarin.Forms;

namespace Fitness_Appv2.Views
{
    public partial class Xcises : ContentPage
    {
        List<XcisesTable> xcisesList;
        int currComponentIndex;[95]
        public Xcises()
        {
            InitializeComponent();
        }
        protected override async void OnAppearing()
        {
            base.OnAppearing();
            xcisesList = await App.Db.GetXcisesAsync(); // needed since itemssource is IEnumerable not List
            xcisesView.ItemsSource = xcisesList;
        }
        private async void SubmitXcise_ClickedAsync(object sender, EventArgs e){
            if (submitXcise.Text == "Submit Exercise")[96]
            {
```

[95] Data Structures
[96] Defensive Programming

```
            string name = xciseNameInput.Text;
            bool isBodyweight = isBodyweightInput.IsChecked;
            List<string> existingXcises = (await App.Db.GetXciseNamesAsync()).Select(obj => obj.XciseNameAttribute).ToList();
            if (!string.IsNullOrEmpty(name) && !existingXcises.Contains(name))
            {
                submitXcise.Text = "Submitted";
                await App.Db.SaveXciseAsync(new XcisesTable
                {
                    XciseNameAttribute = name,
                    IsBodyweightAttribute = isBodyweight,
                });

                xcisesList = null;
                xcisesList = await App.Db.GetXcisesAsync();

                xcisesView.ItemsSource = null;
                xcisesView.ItemsSource = xcisesList;

                xciseNameInput.Text = "";
                isBodyweightInput.IsChecked = false;
                await Task.Delay(1500);
                submitXcise.Text = "Submit Exercise";
            }
        }
    }

    private void XcisesView_SelectionChanged(object sender, SelectionChangedEventArgs e)
    {
        currComponentIndex = xcisesList.IndexOf(e.CurrentSelection[0] as XcisesTable);
```

[97] Defensive Programming
[98] Data Structures

```
            isPinnedInput.IsChecked = xcisesList[currComponentIndex].IsPinnedAttribute;
            changeIsPinned.IsVisible = true;
        }

        private void SubmitIsPinned_Clicked(object sender, EventArgs e)
        {
            xcisesList[currComponentIndex].IsPinnedAttribute = isPinnedInput.IsChecked;[99]
            App.Db.UpdateXciseIsPinnedAsync(XcisesList[currComponentIndex].IsPinnedAttribute, xcisesList[currComponentIndex].Id);
            xcisesView.ItemsSource = null;
            xcisesView.ItemsSource = xcisesList;
            changeIsPinned.IsVisible = false;
        }
    }
}
```

Profile.xaml

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="Fitness_Appv2.Views.Profile">

    <Grid RowDefinitions="Auto, Auto, Auto, Auto, Auto, Auto, Auto, Auto">

        <Label x:Name="BodyweightDisplay" Grid.Row="0"/>
        <Entry x:Name="BodyweightInput" Keyboard="Numeric" Placeholder="Input Bodyweight" Grid.Row="1"/>
        <Button x:Name="BodyweightUpdate" Pressed="BodyweightUpdate_Clicked" Text="Update" Grid.Row="2"/>

        <Label x:Name="ConsistencyDisplay" Grid.Row="4"/>

        <Label x:Name="ConsistencyGoalDisplay" Grid.Row="5"/>
```

[99] Data Structures

```
        <Entry x:Name="ConsistencyGoalInput" Keyboard="Numeric" Placeholder="Input Consistency Goal" Grid.Row="6"/>
        <Button x:Name="ConsistencyGoalUpdate" Pressed="ConsistencyGoalUpdate_Clicked" Text="Update" Grid.Row="7"/>
    </Grid>
</ContentPage>
```

Profile.xaml.cs

```
using Fitness_Appv2.Services;
using System;
using Xamarin.Forms;
using System.Collections.Generic;
using System.Linq;

namespace Fitness_Appv2.Views
{
    public partial class Profile : ContentPage
    {
        public Profile()
        {
            InitializeComponent();
        }

        protected override void OnAppearing()
        {
            base.OnAppearing();
            DisplayUserDataAsync();
        }

        public async void DisplayUserDataAsync()
        {
            List<UserDataTable> userdata = await App.Db.GetUserDataAsync(); // get user data orders by date descending
            UserDataTable latest = userdata[0];
```

```
            // no need to check for null as CheckNeedNewUserData is called (sets bodyweight & goal = 0 on null)
            if (latest.BodyweightAttribute != 0)
            {
                BodyweightDisplay.Text = "Current Bodyweight: \n" + latest.BodyweightAttribute;
            }
            else
            {
                BodyweightDisplay.Text = "Current Bodyweight: \nNot Set";
            }
            if (latest.WeeklyConsistencyGoalAttribute != 0)
            {
                ConsistencyGoalDisplay.Text = "Current Goal: \n" + latest.WeeklyConsistencyGoalAttribute;
            }
            else
            {
                ConsistencyGoalDisplay.Text = "Current Goal: \nNot Set";
            }

            DateTime lastMonday = Utilities.GetLastMonday();
            UserDataTable userdataLastWeek;
            try
            {
                userdataLastWeek = userdata.Where(obj => obj.DateAttribute == lastMonday.AddDays(-7)).ToList()[0];
            }
            catch
            {
                userdataLastWeek = null;[100]
            }
```

[100] Exception Handling

```
            if (userdataLastWeek != null)
            {
                if (userdataLastWeek.WeeklyConsistencyGoalAttribute != 0)
                {
                    ConsistencyDisplay.Text = "Last Week's Consistency: \n" + userdataLastWeek.WeeklyConsistencyAttribute
                    + " out of " + userdataLastWeek.WeeklyConsistencyGoalAttribute;
                } else
                {
                    ConsistencyDisplay.Text = "Last Week's Consistency: \n" + userdataLastWeek.WeeklyConsistencyAttribute;
                }
            }
            else
            {
                if (userdataLastWeek != null && userdataLastWeek.WeeklyConsistencyGoalAttribute != 0)
                {
                    ConsistencyDisplay.Text = "Last Week's Consistency: \n0" + " out of " + userdataLastWeek.WeeklyConsistencyGoalAttribute;
                }
                else
                {
                    ConsistencyDisplay.Text = "Last Week's Consistency: \n0";
                }
            }

        }


        private void BodyweightUpdate_Clicked(object sender, EventArgs e) // input string not in right format
        {
            float bodyweight = Convert.ToSingle(BodyweightInput.Text);
            if (bodyweight > 0)[101]
```

[101] Defensive Programming

```
            {
                App.Db.UpdateBodyweightThisWeekAsync(bodyweight);

                BodyweightInput.Text = "";
                DisplayUserDataAsync();
            }
        }

        private void ConsistencyGoalUpdate_Clicked(object sender, EventArgs e) // input string not in right format
        {
            int consistencyGoal = Convert.ToInt16(ConsistencyGoalInput.Text);
            if (consistencyGoal > 0)[102]
            {
                App.Db.UpdateConsistencyGoalThisWeekAsync(consistencyGoal);

                ConsistencyGoalInput.Text = "";
                DisplayUserDataAsync();
            }
        }
    }
}
```

[102] Defensive Programming

Testing

Objectives Repeated

Stage I

1. App works for android phones.
2. User can navigate intuitively between different pages of the app.
3. Tables for Pending Sets, Monthly Medians and Daily Medians implemented using an SQL Database.
4. User can store a set by inputting the weightlifting exercise, weight used, and repetitions (reps).
5. Based off the user input for a set, the app stores locally the type of exercise, date and a prediction for how much weight that person could lift for 1 repetition (rep), as a marker for how powerful that set was (referred to as estimated 1 rep max or e1RM).
6. User can view sets from the last month in a table.
7. Database stores, in a separate table, medians of e1RM, to smooth out the graphs and make trends more obvious.
    a) For each exercise, the app stores the median e1RM for sets on the same day, on days this month.
    b) For each exercise, the app stores the median e1RM for sets in the same month, once that month has ended.
8. User can view graphs of median e1RM (both daily and monthly) against Time for a given exercise.

Stage II

9. Implement tables for User Data, Exercises, Routines, and Routine Components in the previously mentioned SQL database.
10. User can record their bodyweight in the app.
11. App stores weight of user over time.
12. User can define custom exercises, inputting their name and whether they are bodyweight exercises or not.
13. App accommodates the recording of bodyweight exercises by automatically adding the stored weight of user to the weight inputted.
14. User can define their own workout routines, by inputting a list of exercises and the number of sets for each of those exercises.
15. User can log user-defined workout routines quickly, by loading up a workout then inputting, for each set, the number of reps and the weight.
16. When logging a predefined workout, the User can deviate from their defined workout by adding or removing exercises and/or sets on the log workouts page, before they log the workout.

Stage III

17. App can generate a goal for each exercise for the next 3 months, based off their performance in that exercise over the last 3 months.
18. App stores a personal best (PB) e1RM for each exercise.
19. User can pin an exercise's PB and Goal to the welcome page.

20. User can view information about each exercise in a table with the columns: Exercise, PB, Goal and Is Pinned.
21. User can undo/redo the adding of sets recorded in the current session
22. User can define a goal for weekly consistency, measured in total sets per week (across all exercises)
23. User can compare last week's consistency to the goal set for that week.
24. User can view graphs of their bodyweight and weekly consistency over time.
25. App works for IOS.

Test Plan

Test	Objective(s)	Purpose	Description	Expected Result	Result Achieved?
I	1	Ensure the app works for android phones.	Attempt to boot up the app on an android emulator / tested throughout.	Boots up as expected / functions as expected.	Yes
II	2	Ensure the app is easily navigable.	Tested throughout.	Functions as expected	Yes
III	3, 4, 5, 6, 7, 8	Ensure it is possible for recent sets to be saved (and have their estimated 1RMax in the database and have their medians taken on a daily and monthly basis.	Add a set in add sets. Check the recent sets display and check graph for medians. Test Data: Yesterday: - 0.5 x 85 --> 79.308 (= 85 * 0.5^0.1) - 5 x 75 --> 84.375 (= 75 * (36/(37-5))) - 20 x 60 --> 80.957 (= 60 * 20^0.1) Daily Median = 80.957 Last Month: - 1 x 78 --> 78 - 2 x 82 --> 82 Monthly Median = 80	Recent sets table should display the inputted set (with the **correct e1RMax, see test data**), graph should pass through pre- calculated points.	Yes
IV	9, 10, 11	Ensure the app can be used to store the user's bodyweight.	Store a new value for bodyweight in the Profile page.	The new bodyweight appears in the display box.	Yes
V	9, 12	Ensure the app can be used to store data about custom exercises that are inputted by the user.	Add an exercise in the exercises page and check the exercises display table.	The exercises display table should show the new exercise.	Yes
VI	13	Ensure the app accommodates use of bodyweight exercises, by calculating e1RMax using the weight inputted plus the user's bodyweight.	Add a set of a bodyweight exercise and check the recent sets display. Test Data: 1 x (10 + 81) --> 91	The recent sets display should show a e1RMax higher than the weight inputted.	Yes

VII	14	Ensure the user can use the app to define their own workout routines.	Create a new workout routine and then check it is saved by exiting and editing.	The routine should be the same after saving and editing.	Yes
VIII	15, 16	Ensure the user can log their own workout routines.	Log an altered version of the original workout routine and check the recent sets display.	The logged routine should match what is in the recent sets with the correct e1RMaxes.	Yes
IX	17, 20	Ensure the app can generate goals for each exercise and that the user can view them.	Check goal in the exercises display on the exercises page then check trend on graph. Test Data - 3 Months Ago: 70 - 2 Months Ago: 80 - 1 Month Ago: 90 3 Month Goal: 130 (= 90 + 4*10)	The goal should align with the approximate trend of the graph.	Yes
X	18, 20	Ensure the app can take a PB for each exercise and that the user can view them.	Input a set with higher estimated 1RMax than the current PB for that exercise then check the exercises display.	The PB for the exercise recorded should change.	Yes
XI	19, 20	Ensure the user can pin exercises to the welcome page and view which exercises are pinned.	Pin a set in the exercises page using the exercises display, then check the welcome page.	The exercise should be pinned, and the exercises display should state that.	Yes
XII	21	Ensure the user can undo/redo sets.	Add several sets, undo some of them and redo a couple Add a set then try and redo.	Follow along with a diagram and check it works as expected. Check redo doesn't work immediately after adding a set.	Yes
XIII	22	Ensure the user can set a goal for weekly consistency.	Set a new weekly consistency goal.	The change should be reflected in the profile page.	Yes
XIV	23	Ensure the user can use their previous week's weekly consistency goal and actual value to review that week.	Keep a paper log of workouts in the last week and the consistency goal, then check the weekly consistency for that week in the profile page.	The sum of the sets should equal the weekly consistency for the last week and the goals should be the same irrespective of the current goal.	Yes
XV	24	Ensure the user can view graphs of consistency and bodyweight.	Check the statistics page and select consistency then bodyweight.	The graph data should now correspond to the consistency /	Yes

				bodyweight of the user in previous weeks. Change the bodyweight in profile page, then look at the graph.	
XVI	25	Ensure the app works for IOS phones.	Boot up the app on an IOS emulator.	Boots up as expected.	No

Evaluation

User Feedback

Mr. E Clegg and P Kiernan are consistent users of the school gym and have tried other fitness apps. After downloading my app and using it for 3 weeks, I interviewed them simultaneously (see below)

Interview with Mr. E Clegg and P Kiernan

How did you find the user interface of the app?

EC: "A bit confusing and occasionally unresponsive but other than that it was good."

PK: "Good layout. Yeah, very clear. What else? Pretty intuitive as well, has everything you would need and a lot of menu bars."

EC: "Some of the data could have been represented better though."

What did you think about the different functions of the app? Would you find them useful?

EC: "Yeah, I think I would find them useful, but I think you might need a bit more if you really wanted to like distinguish your app from others."

PK: "I thought it was good though; I actually really liked the versatility of like the graph section of the app."

Was there anything you feel could be added to this app in a future version?

PK: "Yeah, I think it could have some like dietary advice for like so that if you're doing loads and loads of lifting in a day, it says, right, you need high calories."

EC: "Just make it look pretty I reckon."

Key Takeaways from the User Interview

- UI needs further development
- Perhaps need a tutorial of some kind (features were intuitive to some but not others)
- Streamlining of features to make them both practical and distinct from other apps
- Broaden capabilities of app e.g. dietary advice

Assessment of my Objectives

Objective	Assessment
1.	Fully Achieved
2.	Mostly achieved, would add a tutorial to better explain app.
3.	Fully Achieved

4.	Fully Achieved
5.	Fully Achieved, would extend objectives to retain the reps and weight as it turns out the capacity of SQLite is a lot bigger than I thought it would be.
6.	Mostly Achieved, would round values when displaying them for readability.
7. a)	Fully Achieved
7. b)	Fully Achieved
8.	Fully Achieved
9.	Fully Achieved
10.	Fully Achieved
11.	Fully Achieved
12.	Fully Achieved, would extend objectives to allow user to delete exercises as well.
13.	Mostly Achieved, would make it clearer that bodyweight is automatically added and that the weight entry field is just for any additional weight.
14.	Mostly Achieved, would allow user to change order of routine components.
15.	Fully Achieved
16.	Mostly Achieved, would allow user to change order of routine components.
17.	Adequately Achieved, would find a way to exclude 0 values from the predictions. Would also allow user to change the timeframes of the predictions.
18.	Fully Achieved, would explain that PB is only updated at midnight.
19.	Fully Achieved
20.	Fully Achieved
21.	Fully Achieved
22.	Fully Achieved
23.	Fully Achieved
24.	Fully Achieved
25.	Not Achieved, would investigate possibility of downloading the app on an iPhone

Conclusion

Overall, I feel very satisfied with what I accomplished with the time that I had, and I feel like I have an app which is very useable and provides features that I haven't seen elsewhere such as the graphing element and the predictive goals.

Because of time constraints, I was forced to follow the critical path of the app, meaning the UI could have been a lot clearer and neater.

In terms of objectives, as you can see from my testing table, I have completed all objectives I set out to complete originally but I haven't been able to test the objective that it works for IOS.

Additionally, since the main objectives were so time consuming to program, there are many features that I would look to include in a future update of the app, including:

- Making it work for IOS (should be simple in theory, but I had technical difficulties surrounding getting access to an IOS emulator)
- Changing the goal generation so 0 values for monthly medians aren't included in the prediction.
- Improving the UI, make it look neater and less clunky.
- Would explain certain quirks of the app better e.g. tutorial to explain the different pages, weight field for bodyweight exercises is just for additional weight, PB is updated at midnight.
- Greater control of statistical analysis e.g. retains original reps and weight inputted and allow them to change time frames of goals.
- Being able to change the order of the routine components.

- Being able to delete exercises
- Showing rounded values of e1RMax in the display.
- Expanding the app to include cardio exercises.
- Expanding the app to include data based dietary advice.

Having said that, I do feel like I included all the necessary features for a data-driven fitness app and will continue to use the app on my phone for personal use.

Acknowledgements

I used the Xamarin Forms flyout template to get started on my project since I quickly found that it would take too much time to program the navigation myself. I have clearly labelled with comments in my implementation any areas that were kept in from the template.

I discovered the Syncfusion component for charts and graph visualisation using a Microsoft Dev Blog (Montemagno 2015) and learnt the basics of how it worked. The code in my listing is all my own however and I have not lifted anything directly from the blog.

References

Wikipedia (2024) *One Repetition Maximum*. Available at: https://en.wikipedia.org/wiki/One-repetition_maximum (Accessed: May 2024).

Geeks for Geeks (2024) *Linear Regression Formula*. Available at: https://www.geeksforgeeks.org/linear-regression-formula (Accessed: December 2024).

Montemagno, James (2015) *Visualize Your Data with Charts, Graphs, and Xamarin.Forms.* Available at: https://devblogs.microsoft.com/xamarin/visualize-your-data-with-charts-graphs-and-xamarin-forms/ (Accessed: July 2024)